# Workflow Templates — Wyniki Testów

Dokument zapisuje wyniki ręcznego testowania każdego szablonu workflow krok po kroku. Każdy test przechodzi pełną ścieżkę: trigger → getter → flow_control → action → create_task.

**Legenda:**

- 🤖 **Auto** — wartość generowana przez system (daty dynamiczne, dane z getterów, wyrażenia)
- 👤 **User** — wartość którą użytkownik musi uzupełnić przed aktywacją (connection_id, project_id, globals, agent_id)
- 📋 **Template** — wartość zakodowana w szablonie, stała, nie wymaga zmiany
- ✅ węzeł działa poprawnie
- ⚠️ węzeł działa ale z zastrzeżeniem
- ❌ węzeł nie działa / błąd

---

## Test #1 — `cpa_campaign_alert`

**Data testu:** 2026-03-27

### Węzeł 1 — Trigger

- 📋 `frequency: weekly`
- 📋 `days_of_week: monday`
- 📋 `time: 08:00`
- 👤 `timezone` — użytkownik może zmienić (domyślnie `Europe/Warsaw`)

**Wynik:** workflow startuje w poniedziałek o 08:00 ✅

---

### Węzeł 2 — Getter: `get_google_campaign`

- 👤 `connection_id` — użytkownik uzupełnia (połączenie Google Ads)
- 👤 `project_id` — użytkownik uzupełnia
- 📋 `campaign_statuses: ["ENABLED"]` — tylko aktywne kampanie

**Dane (zmyślone, format CampaignDTO):**

```
[
  { "campaignId": "1001", "campaignName": "Brand - Exact", "biddingStrategyType": "MAXIMIZE_CLICKS" },
  { "campaignId": "1002", "campaignName": "Non-Brand - Shoes", "biddingStrategyType": "TARGET_CPA" },
  { "campaignId": "1003", "campaignName": "Remarketing - All Visitors", "biddingStrategyType": "TARGET_CPA" },
  { "campaignId": "1004", "campaignName": "Shopping - All Products", "biddingStrategyType": "TARGET_ROAS" }
]
```

**Wynik:** 4 kampanie przekazane dalej ✅

---

### Węzeł 3 — flow_control: `iterator`

- 📋 `sourceId: 2`

**Wynik:** pętla po 4 kampaniach, każda przechodzi osobno ✅

---

### Węzeł 4 — Getter: `get_google_ads_metrics`

- 👤 `connection_id` — użytkownik uzupełnia
- 👤 `project_id` — użytkownik uzupełnia
- 🤖 `resourceId: {3.campaignId}` — ID kampanii z iteratora
- 🤖 `start_date: {NOW().subtractNumberOfDays(30).formatDateTime('Y-m-d')}` — 30 dni temu

- 🤖 `end_date: {NOW().formatDateTime('Y-m-d')}` — dziś
- 📋 `limit: 1`

**Dane (zmyślone, format MetricsDTO) — ostatnie 30 dni:**

| Kampania | cost | clicks | conversions | CPA (cost/conv) | impressions |
|---|---|---|---|---|---|
| Brand - Exact | 980 | 1240 | 0 | ∞ | 22000 |
| Non-Brand - Shoes | 3400 | 2100 | 31 | 109.68 | 67000 |
| Remarketing | 540 | 320 | 8 | 67.50 | 41000 |
| Shopping | 2100 | 890 | 44 | 47.73 | 95000 |

**Wynik:** metryki pobrane dla każdej kampanii ✅

---

## Węzeł 5 — flow_control: `filter`

- 👤 `globals.min_spend: 100` — użytkownik ustawia minimalny wydatek
- 👤 `globals.cpa_target: 50` — użytkownik ustawia cel CPA

**Warunek:** `cost > min_spend` AND `cpa > cpa_target`

| Kampania | cost > 100? | CPA > 50? | Przechodzi? | Uwaga |
|---|---|---|---|---|
| Brand - Exact | 980 ✅ | ∞ ✅ | ✅ | ⚠️ conversions=0, CPA=∞ — fałszywy alarm |
| Non-Brand - Shoes | 3400 ✅ | 109.68 ✅ | ✅ | słuszny alert |
| Remarketing | 540 ✅ | 67.50 ✅ | ✅ | słuszny alert |
| Shopping | 2100 ✅ | 47.73 ❌ | ❌ | odfiltrowana |

**Wynik:** 3 kampanie przechodzą, w tym 1 fałszywy alarm ✅/⚠️

---

## Węzeł 6 — flow_control: `iterator`

**Wynik:** pętla po 3 przefiltrowanych kampaniach ✅

---

## Węzeł 7 — action: `create_task`

- 🤖 `title` — generowany z wyrażeń dynamicznych
- 🤖 `description` — generowany z wyrażeń dynamicznych
- 👤 `project_id` — użytkownik uzupełnia

**Task #1 — Brand - Exact (fałszywy alarm):**

> ***Tytuł:*** ⚠️ *`CPA Alert: Brand - Exact — CPA: ∞ (cel: 50)`*

> ***Opis:*** *Kampania Brand - Exact przekroczyła docelowy CPA. CPA: ∞ | Cel: 50 | Koszt: 980 | Konwersje: 0 | Kliknięcia: 1240*

**Task #2 — Non-Brand - Shoes:**

> ***Tytuł:*** ⚠️ *`CPA Alert: Non-Brand - Shoes — CPA: 109.68 (cel: 50)`*

> ***Opis:*** *Kampania Non-Brand - Shoes przekroczyła docelowy CPA. CPA: 109.68 | Cel: 50 | Koszt: 3400 | Konwersje: 31 | Kliknięcia: 2100*

**Task #3 — Remarketing - All Visitors:**

> ***Tytuł:*** ⚠️ *`CPA Alert: Remarketing - All Visitors — CPA: 67.50 (cel: 50)`*

> ***Opis:*** *Kampania Remarketing - All Visitors przekroczyła docelowy CPA. CPA: 67.50 | Cel: 50 | Koszt: 540 | Konwersje: 8 | Kliknięcia: 320*

---

## Podsumowanie #1

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 3 (w tym 1 fałszywy alarm) |
| **Znane problemy** | Brak filtra na `biddingStrategyType` lub `conversions > 0` — kampanie bez konwersji jako celu (MAXIMIZE_CLICKS, TARGET_ROAS) generują fałszywe alarmy z CPA=∞ |
| **Rekomendacja** | Dodać warunek `conversions > 0` do filtra |

---

# Test #2 — `roas_drop_alert`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`
- 📋 `days_of_week: monday`
- 📋 `time: 08:00`
- 👤 `timezone` — domyślnie `Europe/Warsaw`

**Wynik:** workflow startuje w poniedziałek o 08:00 ✅

---

## Węzeł 2 — Getter: `get_google_campaign`

- 👤 `connection_id`
- 👤 `project_id`
- 📋 `campaign_statuses: ["ENABLED"]`

**Dane (zmyślone, format CampaignDTO):**

```
[
  { "campaignId": "1001", "campaignName": "Brand - Exact", "biddingStrategyType": "MAXIMIZE_CLICKS" },
  { "campaignId": "1002", "campaignName": "Non-Brand - Shoes", "biddingStrategyType": "TARGET_CPA" },
  { "campaignId": "1003", "campaignName": "Remarketing - All Visitors", "biddingStrategyType": "TARGET_CPA" },
  { "campaignId": "1004", "campaignName": "Shopping - All Products", "biddingStrategyType": "TARGET_ROAS" }
]
```

**Wynik:** 4 kampanie ✅

---

## Węzeł 3 — flow_control: `iterator`

- 📋 `sourceId: 2`

**Wynik:** pętla po 4 kampaniach ✅

---

## Węzeł 4 — Getter: `get_google_ads_metrics` — bieżący tydzień

- 👤 `connection_id`
- 👤 `project_id`
- 🤖 `resourceId: {3.campaignId}`
- 🤖 `start_date: {NOW().subtractNumberOfDays(7).formatDateTime('Y-m-d')}` — 7 dni temu
- 🤖 `end_date: {NOW().formatDateTime('Y-m-d')}` — dziś
- 📋 `limit: 1`

**Dane (zmyślone) — ostatnie 7 dni:**

| Kampania | cost | conversions_value | ROAS (val/cost) | conversions | clicks |
|---|---|---|---|---|---|
| Brand - Exact | 280 | 1960 | 7.00 | 0 | 390 |
| Non-Brand - Shoes | 890 | 1602 | 1.80 | 9 | 540 |
| Remarketing | 140 | 336 | 2.40 | 2 | 88 |
| Shopping | 620 | 2480 | 4.00 | 16 | 230 |

**Wynik:** metryki bieżącego tygodnia pobrane ✅

---

## Węzeł 5 — Getter: `get_google_ads_metrics` — poprzedni tydzień

- 👤 `connection_id`
- 👤 `project_id`
- 🤖 `resourceId: {3.campaignId}`
- 🤖 `start_date: {NOW().subtractNumberOfDays(14).formatDateTime('Y-m-d')}` — 14 dni temu
- 🤖 `end_date: {NOW().subtractNumberOfDays(7).formatDateTime('Y-m-d')}` — 7 dni temu
- 📋 `limit: 1`

**Dane (zmyślone) — poprzednie 7 dni:**

| Kampania | cost | conversions_value | ROAS (val/cost) | conversions |
|---|---|---|---|---|
| Brand - Exact | 260 | 2080 | 8.00 | 0 |
| Non-Brand - Shoes | 810 | 3240 | 4.00 | 18 |
| Remarketing | 130 | 520 | 4.00 | 4 |
| Shopping | 590 | 2360 | 4.00 | 15 |

**Wynik:** metryki poprzedniego tygodnia pobrane ✅

---

## Węzeł 6 — flow_control: `filter`

- 👤 `globals.min_spend: 100`
- 👤 `globals.roas_target: 3.0`

**Warunek:** `{4.cost} > min_spend` AND `{divide({4.conversions_value},{4.cost})} < roas_target`

| Kampania | cost > 100? | ROAS bieżący | ROAS < 3.0? | Przechodzi? | Uwaga |
|---|---|---|---|---|---|
| Brand - Exact | 280 ✅ | 7.00 | ❌ | ❌ | odfiltrowana |
| Non-Brand - Shoes | 890 ✅ | 1.80 | ✅ | ✅ | słuszny alert |
| Remarketing | 140 ✅ | 2.40 | ✅ | ✅ | słuszny alert |
| Shopping | 620 ✅ | 4.00 | ❌ | ❌ | odfiltrowana |

**Wynik:** 2 kampanie przechodzą ✅

⚠️ **Uwaga:** kampanie `TARGET_ROAS` i `MAXIMIZE_CLICKS` mogą generować mylące alerty — ten sam problem co w #1, brak filtra na strategię bidowania.

---

## Węzeł 7 — flow_control: `iterator`

- 📋 `sourceId: 4`

**Wynik:** pętla po 2 przefiltrowanych kampaniach ✅

---

## Węzeł 8 — action: `create_task`

- 🤖 `title` — dynamiczny z ROAS i nazwą kampanii
- 🤖 `description` — porównanie dwóch tygodni
- 👤 `project_id`

**Task #1 — Non-Brand - Shoes:**

> ***Tytuł:*** *`📉 ROAS Alert: Non-Brand - Shoes — ROAS: 1.80 (cel: 3.0)`*

>

> ***Opis:***

> 🏷️ *Kampania: Non-Brand - Shoes (ID: 1002)*

>

> *📈 Bieżący tydzień (20.03 – 27.03):*

> *Koszt: 890 | Wartość konwersji: 1602 | Konwersje: 9 | ROAS: 1.80 | Kliknięcia: 540*

>

> *📉 Poprzedni tydzień (13.03 – 20.03):*

> *Koszt: 810 | Wartość konwersji: 3240 | Konwersje: 18 | ROAS: 4.00*

>

> *🎯 Cel ROAS: 3.0 | ⚠️ Zmiana ROAS: -55%*

**Task #2 — Remarketing - All Visitors:**

> ***Tytuł:*** `📉 ROAS Alert: Remarketing - All Visitors — ROAS: 2.40 (cel: 3.0)`

>

> ***Opis:***

> *🏷️ Kampania: Remarketing - All Visitors (ID: 1003)*

>

> *📈 Bieżący tydzień (20.03 – 27.03):*

> *Koszt: 140 | Wartość konwersji: 336 | Konwersje: 2 | ROAS: 2.40 | Kliknięcia: 88*

>

> *📉 Poprzedni tydzień (13.03 – 20.03):*

> *Koszt: 130 | Wartość konwersji: 520 | Konwersje: 4 | ROAS: 4.00*

>

> *🎯 Cel ROAS: 3.0 | ⚠️ Zmiana ROAS: -40%*

---

## Podsumowanie #2

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 2 |
| **Znane problemy** | 1. Brak filtra na `biddingStrategyType` / `conversions > 0` — ten sam problem co #1. 2. Nazwa "ROAS Drop Alert" sugeruje wykrywanie spadku, logika wykrywa przekroczenie progu — kampania z ROAS 1.5 przez cały czas nie dostanie alertu dopóki user nie ustawi odpowiedniego targetu. |
| **Rekomendacja** | Rozważyć dodanie warunku na zmianę % ROAS tydzień do tygodnia jako drugi trigger alertu |

---

# Test #3 — `zero_impression_campaigns`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`
- 📋 `days_of_week: monday`
- 📋 `time: 08:00`
- 📋 `timezone: Europe/Warsaw`

**Wynik:** workflow startuje w poniedziałek o 08:00 ✅

---

## Węzeł 2 — Getter: `get_google_campaign`

- 👤 `connection_id`
- 👤 `project_id`
- 📋 `campaign_statuses: ["ENABLED"]`

**Dane (zmyślone, format CampaignDTO) — rozszerzony zestaw o edge case'y:**

```
[
  { "campaignId": "1001", "campaignName": "Brand - Exact", "campaignType": "SEARCH" },
  { "campaignId": "1002", "campaignName": "Non-Brand - Shoes", "campaignType": "SEARCH" },
  { "campaignId": "1003", "campaignName": "Shopping - All Products", "campaignType": "SHOPPING" },
  { "campaignId": "1004", "campaignName": "Display - Remarketing", "campaignType": "DISPLAY" },
  { "campaignId": "1005", "campaignName": "New Campaign - launched 2 days ago", "campaignType": "SEARCH" },
  { "campaignId": "1006", "campaignName": "Weekend Only - Ad Schedule", "campaignType": "SEARCH" }
]
```

**Wynik:** 6 kampanii ✅

---

## Węzeł 3 — flow_control: `iterator`

- 📋 `sourceId: 2`
- 📋 `label: "Iteracja po aktywnych kampaniach"`

**Wynik:** pętla po 6 kampaniach ✅

---

## Węzeł 4 — Getter: `get_google_ads_metrics`

- 👤 `connection_id`
- 👤 `project_id`
- 🤖 `resourceId: {3.campaignId}` — ID kampanii z iteratora
- 🤖 `start_date: {NOW().subtractNumberOfDays(7).formatDateTime('Y-m-d')}` — 7 dni temu
- 🤖 `end_date: {NOW().formatDateTime('Y-m-d')}` — dziś
- 📋 `limit: 1`

**Dane (zmyślone) — ostatnie 7 dni:**

| Kampania | impressions | clicks | cost | conversions | Uwaga |
|---|---|---|---|---|---|
| Brand - Exact | 18400 | 890 | 1200 | 42 | aktywna |
| Non-Brand - Shoes | 67000 | 2100 | 3400 | 31 | aktywna |
| Shopping - All Products | 0 | 0 | 0 | 0 | problem z feedem |
| Display - Remarketing | 0 | 0 | 0 | 0 | wyczerpany budżet |
| New Campaign (2 dni) | 0 | 0 | 0 | 0 | ⚠️ za młoda |
| Weekend Only | 0 | 0 | 0 | 0 | ⚠️ harmonogram reklam |

**Wynik:** metryki pobrane ✅

---

## Węzeł 5 — flow_control: `filter`

- 📋 `impressions < 1` — zero wyświetleń (nie konfigurowalny przez użytkownika — zero to zero)

**Ewaluacja:**

| Kampania | impressions < 1? | Przechodzi? | Uwaga |
|---|---|---|---|
| Brand - Exact | 18400 ❌ | ❌ | odfiltrowana |
| Non-Brand - Shoes | 67000 ❌ | ❌ | odfiltrowana |
| Shopping - All Products | 0 ✅ | ✅ | słuszny alert |
| Display - Remarketing | 0 ✅ | ✅ | słuszny alert |
| New Campaign (2 dni) | 0 ✅ | ✅ | ⚠️ fałszywy alarm — kampania nowa |
| Weekend Only | 0 ✅ | ✅ | ⚠️ potencjalnie fałszywy — zależnie od dnia testu |

**Wynik:** 4 kampanie przechodzą, w tym 1–2 potencjalnie fałszywe alarmy ✅/⚠️

---

## Węzeł 6 — flow_control: `iterator`

- 📋 `sourceId: 4`

**Wynik:** pętla po 4 przefiltrowanych kampaniach ✅

---

## Węzeł 7 — action: `create_task`

- 🤖 `title` — dynamiczny z nazwą kampanii i datą
- 🤖 `description` — lista możliwych przyczyn z datami okresu
- 👤 `project_id`

**Task #1 — Shopping - All Products:**

> ***Tytuł:*** *`👻 Zero wyświetleń: Shopping - All Products (ostatnie 7 dni)`*

>

> ***Opis:***

> *📊 Zero Impression Audit — 27.03.2026*

> *🏷️ Kampania: Shopping - All Products (ID: 1003)*

> *📈 Wyniki 20.03 – 27.03: Wyświetlenia: 0 | Kliknięcia: 0 | Koszt: 0 | Konwersje: 0*

> ⚠️ *Kampania ma status ENABLED, ale nie uzyskała żadnych wyświetleń.*

> 🔍 *Możliwe przyczyny: budżet, stawki CPC, status konta, targetowanie, słowa kluczowe, grupy reklam*

> 💡 *Następny krok: otwórz kampanię i sprawdź kartę 'Diagnostyka kampanii'*

**Task #2 — Display - Remarketing:** *(analogiczny format)*

**Task #3 — New Campaign:** *(analogiczny format — fałszywy alarm)*

**Task #4 — Weekend Only:** *(analogiczny format — potencjalnie fałszywy)*

---

## Pytania i wnioski z testu

**Czy workflow obsługuje nowe kampanie?** Nie — brak warunku na wiek kampanii. Kampania uruchomiona 2 dni temu z zerowym ruchem dostanie alert identyczny jak kampania mająca problem od tygodnia. Użytkownik traci czas na sprawdzanie czegoś co jest normalnym stanem nowej kampanii.

**Czy harmonogram reklam (ad scheduling) jest obsługiwany?** Nie — kampania ustawiona na "tylko weekendy" sprawdzana w tygodniu powszednim da zero wyświetleń i wygeneruje fałszywy alarm. Getter nie zwraca konfiguracji harmonogramu reklam.

**Czy to poważny problem?** Oba przypadki to edge case'y — w typowym koncie e-commerce większość kampanii ENABLED powinna mieć wyświetlenia przez 7 dni. Alert pozostaje użyteczny, ale opis szablonu powinien ostrzegać użytkownika o tych sytuacjach.

---

## Podsumowanie #3

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 4 (2 słuszne, 2 potencjalnie fałszywe) |
| **Znane problemy** | 1. Brak filtra na wiek kampanii — nowe kampanie generują fałszywe alarmy. 2. Brak obsługi ad schedulingu — kampanie "tylko weekendy" alarmują w dni powszednie. |
| **Ocena ogólna** | Jeden z lepszych szablonów — prosta jednoznaczna logika, dobry opis z listą przyczyn do sprawdzenia. Fałszywe alarmy możliwe ale rzadkie w typowym koncie. |
| **Rekomendacja** | Dodać do opisu szablonu wzmiankę o nowych kampaniach i harmonogramie reklam jako znanych źródłach fałszywych alarmów |

---

# Test #4 — `budget_protector`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `trigger_type: cron`
- 📋 `cron: "0 "` — co godzinę, stary format

⚠️ **Stary format triggera.** Brak `timezone`, `start_date`, `end_date` — pola konfiguracyjne nieobecne.

**Wynik:** workflow startuje co godzinę ✅ (technicznie działa, ale patrz wnioski)

---

## Węzeł 2 — Getter: `get_google_campaign`

- 👤 `connection_id` — puste, użytkownik musi uzupełnić
- 👤 `project_id` — puste, użytkownik musi uzupełnić

**Dane (zmyślone, format CampaignDTO):**

```
[
  { "campaignId": "1001", "campaignName": "Brand - Exact" },
  { "campaignId": "1002", "campaignName": "Non-Brand - Shoes" },
  { "campaignId": "1003", "campaignName": "Shopping - All Products" }
]
```

---

## Węzeł 3 — flow_control: `iterator`

- 📋 `sourceId: 2`

**Wynik:** iteruje po 3 kampaniach — 3 przebiegi ✅

---

## Węzeł 4 — Getter: `get_google_ads_metrics` (per kampania)

- 👤 `connection_id` — puste
- 👤 `project_id` — puste
- 🤖 `resourceId: {3.campaignId}` — z iteratora
- 🤖 `startDate: {NOW().subtractNumberOfDays(1).formatDateTime('Y-m-d')}` — wczoraj (26.03.2026)
- 🤖 `endDate: {NOW().formatDateTime('Y-m-d')}` — dziś (27.03.2026)

⚠️ **Zakres dat: wczoraj + dziś.** Przy uruchomieniu o 09:00 pobiera dane od wczoraj. Dzisiejsze dane są niepełne — koszt narastający, nie całodniowy. To sprawia że porównanie z progiem `cost > 100` jest problematyczne rano (niski koszt) a może trafiać wieczorem.

**Dane:**

```
{ "cost": 145.20, "clicks": 87, "impressions": 1240, "conversions": 3.2 }
{ "cost": 67.50, "clicks": 42, "impressions": 830, "conversions": 1.1 }
{ "cost": 210.00, "clicks": 156, "impressions": 2100, "conversions": 8.4 }
```

---

## Węzeł 5 — flow_control: `filter`

- 📋 `left: "cost"`, `operator: ">"`, `right: "100"` — hardcoded próg 100 (waluta?)
- 📋 `impressions > 0` — drugi warunek

⚠️ **Próg 100 jest hardcoded i nie ma konfigurowalnego global.** Brak informacji o walucie. Dla konta w EUR próg 100 = ~430 PLN. Dla konta z budżetem 1000 PLN/dzień próg 100 PLN to 10% — dla konta z budżetem 10 000 PLN/dzień to 1%. Ten sam szablon jest nieadekwatny dla kont różnej skali.

**Wynik filtra:** kampanie 1001 (145.20 > 100 ✅) i 1003 (210.00 > 100 ✅) przechodzą. Kampania 1002 (67.50 ≤ 100 ❌) odpada.

---

### Węzeł 6 — action: `create_task`

- 📋 `name: "Budget alert: {3.campaignName}"`
- 📋 `description: "Campaign {3.campaignName} has spent {4.cost} today..."`
- ❌ Brak `project_id` w payloadzie — użytkownik nie ma gdzie go ustawić. Starszy format (`action_type: create_task` w `data`).
- ❌ `globals: []` — pusta tablica zamiast obiektu, brak konfigurowalnych wartości

**Task #1 (kampania Brand):**

> ***Tytuł:*** `Budget alert: Brand - Exact`

> ***Opis:*** *Campaign Brand - Exact has spent 145.2 today. Clicks: 87, Conversions: 3.2*

**Task #2 (kampania Shopping):**

> ***Tytuł:*** `Budget alert: Shopping - All Products`

> ***Opis:*** *Campaign Shopping - All Products has spent 210 today. Clicks: 156, Conversions: 8.4*

---

## Pytania i wnioski z testu

**Ile tasków może wygenerować w ciągu doby?** Trigger co 1 godzinę = 24 uruchomienia/dobę. Jeśli kampania przekracza próg przez cały dzień → 24 taski per kampania. Przy 10 kampaniach powyżej progu → 240 tasków/dobę. To jest "task flood" — lista zadań zostaje zaśmiecona tym samym alertem.

**Dlaczego brak konfigurowalnego progu to problem?** Prog 100 jest odpowiedni dla małego konta e-commerce, ale absurdalnie niski dla średniej agencji z budżetami 5000+ PLN/dzień. Szablon jest nieużywalny bez modyfikacji dla większości profesjonalnych kont.

**Czy można naprawić bez przebudowy?** Nie. Wymagane zmiany: (1) nowy format triggera, (2) global `daily_budget_threshold`, (3) interval co 4h zamiast 1h, (4) mechanizm deduplikacji lub warunek "alert tylko raz dziennie".

---

## Podsumowanie #4

| | |
|---|---|
| **Status** | ❌ błędny projekt — nie nadaje się do wdrożenia |
| **Taski utworzone** | 2 per uruchomienie × 24 uruchomienia = 48 tasków/dobę dla tego przykładu |
| **Znane problemy** | 1. Trigger co 1h bez deduplikacji = task flood. 2. Hardcoded próg `cost>100` nieadekwatny dla różnych kont. 3. Stary format (cron trigger, action_type w data). 4. `globals:[]` — brak konfiguracji. 5. Brak `project_id` w create_task. |
| **Ocena ogólna** | Jeden z najgorszych szablonów — zdolny do zaśmiecenia task listy setkami duplikatów. Dobra idea (monitoring budżetu w ciągu dnia), fatalna implementacja. |
| **Rekomendacja** | Przebudować wzorując się na `ms_budget_protector` — nowy format, global threshold, interval 4h |

---

# Test #5 — `budget_deviation`

**Data testu:** 2026-03-27

## ⚠️ UWAGA: To nie jest szablon — to prawdziwy workflow użytkownika

Plik zawiera:

- `creator.email: lukas.lochocki@sembot.io` — email pracownika Sembot
- `project_id: 909` — hardcoded w wielu węzłach
- `connection_id: 652` — hardcoded w węzłach getter
- `configured: true`, `start_date: 2025-12-19`
- Filtr z hardcoded ID klienta Google Ads: `"right_operand_value": "8725857420"`

**Problem GDPR:** email pracownika w publicznym pliku asset załadowanym przez wszystkich użytkowników Sembot. Musi zostać zastąpiony prawdziwym templatem.

---

## Węzeł 1 — Trigger

- 📋 `frequency: daily`, `time: "20:00"`, `days_of_month: "first_day"`

⚠️ Uruchamia się **tylko pierwszego dnia miesiąca** (`days_of_month: "first_day"`). Sprawdzanie odchylenia budżetu tylko raz w miesiącu to za rzadko — sens ma sprawdzanie dziennie lub co kilka dni.

---

## Węzeł 4 — Getter: `get_google_ads_customer`

- ❌ `connection_id: 652` — hardcoded ID połączenia użytkownika
- ❌ `project_id: 909` — hardcoded ID projektu

---

## Węzeł 8 — flow_control: `filter`

- ❌ `id = "8725857420"` — hardcoded ID klienta Google Ads

Ten filtr przepuszcza wyłącznie konto Google Ads o ID 8725857420. Każdy inny użytkownik aktywujący ten "szablon" dostanie pusty wynik — zero danych, zero tasków, workflow nie robi nic.

---

## Węzeł 9 — flow_control: `router`

Router rozdziela na dwie gałęzie: "Budżet przekroczył 10%" i "Budżet poniżej 10%".

**Formuła (gałąź 1 — przekroczenie):** `cost > monthlyDeviation(55000, 10)`

Dla 27 dnia marca (31 dni):

- Oczekiwany koszt do 27.03: `55000 × (27/31) × 1.10 = 52,258 PLN`
- Jeśli koszt rzeczywisty > 52,258 → alert "przekroczenie"

**Formuła (gałąź 2 — niedostateczne wydatki):** `cost < monthlyDeviation(55000, -10)`

- Próg dolny: `55000 × (27/31) × 0.90 = 42,757 PLN`
- Jeśli koszt rzeczywisty < 42,757 → alert "niedowydawanie"

**Dane testowe:**

- `cost: 47,500 PLN` → 42,757 < 47,500 < 52,258 → oba filtry odrzucają → brak taska (budżet w normie ✅)

---

## action: `create_task` (dwie wersje)

Oba taski mają bogaty opis: koszt rzeczywisty vs oczekiwany, % odchylenia, dni do końca miesiąca, rekomendowany dzienny budżet. Treść szablonu jest wzorcowa.

**Przykładowy task (gdyby cost=58,000):**

> ***Tytuł:*** `Budżet na dzień 27 obecnego miesiąca przekroczył 10% odchylenia`

> ***Opis:*** *Miesięczny limit: 55000 zł | Wydatki do 27.03: 58000 zł | Oczekiwane: 47,903 zł | Odchylenie: +21.1% | Pozostały budżet: -3000 zł | Rekomendowany dzienny budżet: -750 zł (ujemny = budżet wyczerpany)*

---

## Podsumowanie #5

| | |
|---|---|
| **Status** | ❌ nie jest templatem — zawiera dane prawdziwego użytkownika |
| **Logika biznesowa** | ✅ wzorcowa — `monthlyDeviation`, router dwugałęziowy, bogaty opis taska |
| **Znane problemy** | 1. GDPR: email pracownika w assets. 2. Hardcoded connection_id/project_id/customer_id → workflow nie działa dla nikogo innego. 3. Trigger tylko 1. dnia miesiąca zamiast codziennie. |
| **Ocena ogólna** | Logika biznesowa zasługuje na 10/10 — to najlepiej napisany workflow w całej kolekcji. Ale jako template jest bezużyteczny. |
| **Rekomendacja** | Przepisać jako prawdziwy template: usunąć hardcoded ID, dodać globals (budget, deviation), zmienić trigger na daily, usunąć filtr po customer ID |

---

# Test #6 — `visibility_drop_alert`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 1"` — stary format, poniedziałek 08:00

---

## Węzeł 2 — Getter: `get_phrase_monitor_competitors`

- 👤 `project_id` — puste

**Dane (zmyślone, format PhrasMonitorCompetitorDTO — snake_case):**

```
[
  { "query": "buty sportowe", "domain": "example.com", "is_my_result": true, "rank_absolute": 3, "url": "example.c
  { "query": "sneakers damskie", "domain": "example.com", "is_my_result": true, "rank_absolute": 8, "url": "exampl
  { "query": "buty do biegania", "domain": "competitor.com", "is_my_result": false, "rank_absolute": 2, "url": "co
  { "query": "adidas running", "domain": "example.com", "is_my_result": true, "rank_absolute": 1, "url": "example.
]
```

---

## Węzeł 3 — flow_control: `filter`

- ❌ `left: "isMyResult"` — camelCase
- ❌ `left: "rankAbsolute"` — camelCase

DTO zwraca pola w `snake_case`: `is_my_result` i `rank_absolute`. Filtr szuka `isMyResult` i `rankAbsolute` — te pola nie istnieją w DTO. Efekt: filtr zachowuje się jak `undefined > 5` i `undefined = true` → wszystkie warunki zawsze fałszywe → ZERO rekordów przechodzi.

**Wynik filtra:**

- Warunek `isMyResult = true`: ❌ pole nie istnieje → warunek fałszywy dla wszystkich → żaden rekord nie przechodzi
- Dane wejściowe: 4 rekordy → dane wyjściowe: 0 rekordów

⚠️ **Błąd krytyczny: filtr nigdy nie zadziała.** Żaden task nigdy nie zostanie stworzony.

---

## Węzeł 5 — action: `create_task`

- 📋 odwołuje się do `{4.query}`, `{4.rankAbsolute}`, `{4.domain}`, `{4.url}`
- ❌ `rankAbsolute` w task description też jest camelCase — po naprawie filtra trzeba też to poprawić

**Task:** *(nigdy nie osiągany)*

---

## Pytania i wnioski z testu

**Czy "visibility drop" to dobra nazwa?** Nie do końca. Workflow nie porównuje dwóch okresów — nie wykrywa SPADKU. Wykrywa obecną pozycję > 5. Pozycja mogła być zawsze > 5, bez żadnego spadku. Bardziej trafna nazwa: "Low Visibility Alert" (obecna widoczność) vs "Visibility Drop Alert" (implikuje porównanie historyczne).

**Czy dane z Phrase Monitor są odpowiednie do tego alertu?** Tak — `is_my_result` i `rank_absolute` to dokładnie te pola których potrzeba. Problem jest wyłącznie w błędnych nazwach pól w filtrze.

---

## Podsumowanie #6

| | |
|---|---|
| **Status** | ❌ błąd krytyczny — filtr zawsze pusty, zero tasków |
| **Taski utworzone** | 0 (zawsze) |
| **Znane problemy** | 1. Pola filtra `isMyResult` / `rankAbsolute` (camelCase) zamiast `is_my_result` / `rank_absolute` (snake_case). 2. Stary format triggera. 3. `rankAbsolute` też w task description. 4. Nazwa "Drop Alert" myląca — brak porównania historycznego. |
| **Ocena ogólna** | Dobry pomysł, wyzerowany przez drobny błąd w nazwach pól. Naprawa jest prosta — zmienić 2 nazwy pól. |
| **Rekomendacja** | Naprawić nazwy pól na snake_case przy migracji do nowego formatu. |

# Test #7 — `price_competitiveness_alert`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: daily`, `time: "07:00"` — nowy format ✅
- 👤 `timezone` — domyślnie `Europe/Warsaw`

---

## Węzeł 2 — Getter: `get_google_merchant_center_price_competitiveness_info`

- 👤 `connection_id` — null, użytkownik uzupełnia
- 👤 `project_id` — null, użytkownik uzupełnia

**Dane (zmyślone, format PriceCompetitivenessDTO — merchant-specific):**

```
[
  { "productViewTitle": "Nike Air Max 90", "productViewBrand": "Nike", "productViewPriceMicros": 45900000, "priceC
  { "productViewTitle": "Adidas Ultraboost", "productViewBrand": "Adidas", "productViewPriceMicros": 55000000, "pr
  { "productViewTitle": "Puma RS-X", "productViewBrand": "Puma", "productViewPriceMicros": 38500000, "priceCompeti
]
```

*(Przypomnienie: 45,900,000 mikros = 45.90 PLN)*

---

## Węzeł 3 — flow_control: `filter`

- 📋 Warunek 1: `priceCompetitivenessBenchmarkPriceMicros > 0` (benchmark istnieje)
- 📋 Warunek 2: `{2.productViewPriceMicros} > {multiply({2.priceCompetitivenessBenchmarkPriceMicros}, {globals.price_premium_threshold})}`
- 👤 `globals.price_premium_threshold` — domyślnie 1.10

Obliczenia dla threshold=1.10:

- Nike Air Max: `45,900,000 > multiply(40,000,000, 1.10)` = `45,900,000 > 44,000,000` → ✅ PRZECHODZI (cena o 14.75% wyżej niż benchmark)
- Adidas Ultraboost: `55,000,000 > multiply(58,000,000, 1.10)` = `55,000,000 > 63,800,000` → ❌ odpada (nasza cena niższa niż benchmark)
- Puma RS-X: `38,500,000 > multiply(32,000,000, 1.10)` = `38,500,000 > 35,200,000` → ✅ PRZECHODZI (cena o 20.3% wyżej niż benchmark)

**Wynik:** 2 produkty (Nike, Puma) przechodzą filtr.

---

## Węzeł 5 — action: `create_task`

**Task #1 — Nike Air Max 90:**

> ***Tytuł:*** *`💸 Cena za wysoka: Nike Air Max 90 – PL`*

>

> ***Opis (fragment):***

> *Produkt: Nike Air Max 90 | Marka: Nike | Kraj: PL*

> *Nasza cena: 45,900,000 PLN | Benchmark: 40,000,000 PLN | Próg: 44,000,000*

> *Nasza cena przekracza benchmark o ponad 10%.*

> *Sugerowane działania: sprawdź marżę, zweryfikuj ceny w GMC → Wzrost i wglądy...*

⚠️ **Wartości w mikrosach w opisie taska.** Użytkownik widzi "45,900,000 PLN" zamiast "45.90 PLN". Szablon ma notatkę o tym w polu description szablonu, ale nie przelicza wartości automatycznie w opisie taska.

---

## Pytania i wnioski z testu

**Czy dane GMC są merchant-specific czy market-level?** `get_google_merchant_center_price_competitiveness_info` zwraca dane **merchant-specific** — porównuje ceny TEGO merchantów z benchmarkiem rynkowym. To wartościowe i unikalne dane, których Google Ads nie ma. ✅

**Czy próg mnożnikowy (1.10) jest dobrym podejściem?** Tak — jest neutralny walutowo i pracuje na mikrosach bez konieczności przeliczania. Matematycznie poprawny.

**Czy problem z mikrosami jest poważny?** Opis szablonu ostrzega użytkownika (nota o ÷1,000,000). Ale task description pokazuje wartości surowe — to może mylić użytkowników którzy nie czytają opisu szablonu.

---

## Podsumowanie #7

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 2 z 3 produktów (Nike, Puma) |
| **Znane problemy** | 1. Ceny w taskach wyświetlane w mikrosach zamiast w walucie konta (mylące dla użytkowników). 2. Szablon uses `inactive` status — wymaga manualnej aktywacji. |
| **Ocena ogólna** | Jeden z lepszych szablonów GMC — merchant-specific dane, konfigurowalny próg, nowy format, dobry opis. Problem z mikrosami jest kosmetyczny (jest nota w opisie). |
| **Rekomendacja** | Rozważyć przeliczanie mikros→waluta w opisie taska przez `{divide({4.productViewPriceMicros}, 1000000)}` |

---

# Test #8 — `inventory_aware_alert`

**Data testu:** 2026-03-27

## ⚠️ UWAGA: Fundamentalny błąd danych

Ten szablon używa `get_google_merchant_center_top_products` — gettera który zwraca **bestsellery całego Google Shopping (market-level)**, NIE produkty merchantów. Oznacza to, że dane dotyczą tysięcy różnych sprzedawców, nie konta użytkownika który aktywował workflow.

---

## Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 *"` — stary format, codziennie 08:00

---

## Węzeł 2 — Getter: `get_google_merchant_center_top_products`

- 👤 `connection_id` — puste
- 👤 `project_id` — puste

**Rzeczywiste pola `BestSellersProductClusterViewDTO`:**

```
productClusterTitle, productClusterBrand, productClusterInventoryStatus,
bestSellersRank, bestSellersRelativeDemandChange, reportCategoryId, reportDate
```

**Pola których NIE MA w DTO** (a których szukają filtr i create_task):

- `conversions` ❌
- `availability` ❌
- `title` ❌
- `offerId` ❌
- `price` ❌

---

## Węzeł 3 — flow_control: `filter`

- ❌ `left: "conversions"` — pole nie istnieje w `BestSellersProductClusterViewDTO`
- ❌ `left: "availability"` — pole nie istnieje (jest `productClusterInventoryStatus`, ale inaczej nazwanego i inaczej kodowanego)

**Wynik filtra:** oba warunki sprawdzają pola które nie istnieją → `undefined > 5` i `undefined contains "out_of_stock"` → zawsze fałszywe → ZERO rekordów przechodzi.

---

## Węzeł 5 — action: `create_task`

- ❌ `name: "Out-of-stock bestseller: {4.title}"` — pole `title` nie istnieje
- ❌ `description` odwołuje się do `{4.title}`, `{4.offerId}`, `{4.availability}`, `{4.conversions}`, `{4.price}` — żadne z tych pól nie istnieje

**Task:** *(nigdy nie osiągany — filtr zawsze pusty)*

---

## Dodatkowy problem: koncepcja nie ma sensu

Nawet gdyby naprawić nazwy pól, workflow ma fundamentalny problem logiczny:

- `productClusterInventoryStatus` w `BestSellersProductClusterViewDTO` to status dostępności u **dowolnego sprzedawcy na świecie**, nie u tego merchantów
- `bestSellersRank` to pozycja w rankingu popularności Google Shopping, nie konwersje merchantów
- Szablon ma sugerować "najlepiej sprzedające się produkty merchantów które są out-of-stock" — ta logika wymaga innego gettera

---

## Podsumowanie #8

| | |
|---|---|
| **Status** | ❌ podwójnie zepsuty — zły getter + nieistniejące pola |
| **Taski utworzone** | 0 (zawsze) |
| **Znane problemy** | 1. Getter zwraca dane market-level, nie produkty merchantów. 2. Filtr i task odwołują się do nieistniejących pól DTO. 3. Stary format triggera. 4. Koncepcja "out-of-stock bestseller" jest niemożliwa do realizacji tym getterem. |
| **Ocena ogólna** | Najgorzej zaprojektowany szablon pod względem poprawności danych — workflow nie może działać w żadnym scenariuszu. |
| **Rekomendacja** | Usunąć. Jeśli zachować ideę — potrzebny getter merchant-specific z danymi inventory per produkt, np. `get_google_merchant_center_performance_info` + osobne źródło stanów magazynowych. |

---

## Test #9 — `non_converting_keywords`

**Data testu:** 2026-03-27

### Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 1"` — stary format, poniedziałek 08:00

---

### Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id` — puste
- 👤 `project_id` — puste
- 📋 `resource: "keyword_view"`
- 🤖 `startDate: {NOW().subtractNumberOfDays(60).formatDateTime('Y-m-d')}` — 26.01.2026
- 🤖 `endDate: {NOW().formatDateTime('Y-m-d')}` — 27.03.2026

✅ Okno 60 dni to dobry kompromis — wystarczająco długo żeby zebrać statystycznie istotne dane, nie za długo żeby nie łapać zarchiwizowanych słów.

**Dane (zmyślone, format keyword_view):**

```
[
  { "title": "buty sportowe tanie", "campaign_name": "Non-Brand", "ad_group_name": "Obuwie tanie", "cost": 45.30,
  { "title": "buty zimowe wyprzedaż", "campaign_name": "Non-Brand", "ad_group_name": "Obuwie zimowe", "cost": 12.5
  { "title": "sneakers nike", "campaign_name": "Brand", "ad_group_name": "Nike", "cost": 67.20, "clicks": 54, "imp
  { "title": "adidas damskie", "campaign_name": "Non-Brand", "ad_group_name": "Adidas", "cost": 28.90, "clicks": 1
]
```

---

### Węzeł 3 — flow_control: `filter`

- 📋 `cost > 20` — hardcoded próg
- 📋 `conversions < 1`

Analiza:

- "buty sportowe tanie": cost=45.30 > 20 ✅, conversions=0 < 1 ✅ → PRZECHODZI
- "buty zimowe wyprzedaż": cost=12.50 ≤ 20 ❌ → odpada
- "sneakers nike": conversions=4 ≥ 1 ❌ → odpada
- "adidas damskie": cost=28.90 > 20 ✅, conversions=0 < 1 ✅ → PRZECHODZI

**Wynik:** 2 słowa kluczowe przechodzą.

⚠️ **Próg `cost>20` jest hardcoded** — brak global. Dla konta z average CPC 15 PLN, słowo które zostało kliknięte raz nie dochodzi do 20 PLN i nie zostanie wykryte. Dla konta premium z CPC 50 PLN próg 20 PLN będzie zawsze przekraczany nawet przez słowa z 1 kliknięciem.

---

### Węzeł 5 — action: `create_task`

**Task #1:**

> ***Tytuł:*** *`Non-converting keyword: buty sportowe tanie`*

> ***Opis:*** *Campaign: Non-Brand, Ad Group: Obuwie tanie, Cost: 45.3, Clicks: 28, Conversions: 0*

**Task #2:**

> ***Tytuł:*** *`Non-converting keyword: adidas damskie`*

> ***Opis:*** *Campaign: Non-Brand, Ad Group: Adidas, Cost: 28.9, Clicks: 19, Conversions: 0*

⚠️ **Skalowalność:** konto z 500 słowami kluczowymi bez konwersji wygeneruje 500 tasków tygodniowo. Lepszy wzorzec: jeden task ze zbiorczą listą słów zamiast task per słowo.

---

## Pytania i wnioski z testu

**Czy 60-dniowe okno jest właściwe dla non-converting keywords?** Tak — dla kampanii z niskim ruchem krótsze okno (7/14 dni) daje za mało danych do oceny. 60 dni pozwala zebrać statystycznie istotną próbę kliknięć bez konwersji.

**Czy filtr `conversions < 1` jest poprawny?** Tak — wartość `0` jest mniejsza niż `1`, więc filtr złapie słowa z dokładnie 0 konwersjami. Prawidłowe.

**Czy brak konwersji zawsze oznacza "złe" słowo?** Nie zawsze — słowa kluczowe dla produktów sezonowych, niedostępnych, lub z bardzo długim cyklem zakupowym mogą mieć 0 konwersji ale realną wartość. Szablon tego nie uwzględnia, ale jest to nieodłączna cecha takiego alertu — użytkownik musi samodzielnie oceniać wyniki.

---

## Podsumowanie #9

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 2 z 4 słów kluczowych |
| **Znane problemy** | 1. Hardcoded `cost>20` — powinien być global. 2. Task per słowo kluczowe = problem skalowalności dla dużych kont. 3. Stary format triggera. |
| **Ocena ogólna** | Logika jest prawidłowa i przydatna. Główny problem to skalowalność — na dużym koncie generuje setki tasków tygodniowo. |
| **Rekomendacja** | Dodać global `min_cost_threshold`. Rozważyć zmianę wzorca na "jeden task z listą słów" zamiast "task per słowo". |

---

# Test #10 — `trending_search_terms`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 1"` — stary format, poniedziałek 08:00

---

## Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id` — puste
- 👤 `project_id` — puste
- 📋 `resource: "search_term_view"`
- 🤖 `startDate`: ostatnie 30 dni
- 📋 `sortField: "impressions"`, `sortOrder: "DESC"`, `limit: 20` — top 20 fraz po wyświetleniach

**Dane (zmyślone, top 20 fraz po wyświetleniach):**

```
[
  { "title": "buty sportowe", "campaign_name": "Non-Brand", "impressions": 4520, "clicks": 312, "ctr": 0.069, "con
  { "title": "sneakers nike air max", "campaign_name": "Brand", "impressions": 2890, "clicks": 245, "ctr": 0.085,
  { "title": "adidas running shoes", "campaign_name": "Non-Brand", "impressions": 1240, "clicks": 87, "ctr": 0.070
  ...
]
```

---

## Węzeł 3 — flow_control: `filter`

- 📋 `impressions > 100`

⚠️ **Filtr `impressions > 100` jest redundantny.** Getter pobiera TOP 20 po `impressions DESC`. Przy typowym koncie e-commerce, żadna z top 20 fraz nie będzie miała < 100 wyświetleń w 30 dni. Filtr przepuszcza praktycznie wszystkie 20 rekordów.

**Wynik:** 20 fraz → ~20 fraz po filtrze (filtr nie eliminuje nic)

---

## Węzeł 5 — action: `create_task`

Tworzy task per fraza — przy 20 frazach = 20 tasków tygodniowo.

**Task #1:**

> ***Tytuł:*** *`Trending search: buty sportowe`*

> ***Opis:*** *Campaign: Non-Brand, Impressions: 4520, Clicks: 312, CTR: 0.069, Conversions: 12*

---

## Pytania i wnioski z testu

**Co tak naprawdę pokazuje ten workflow?** TOP 20 fraz wyszukiwania po liczbie wyświetleń z ostatnich 30 dni. To **popularne frazy**, nie **rosnące (trending) frazy**. Nazwa "Trending Search Terms" jest myląca — implikuje wzrost zainteresowania, a workflow pokazuje popularność bezwzględną.

**Jak wyglądałby prawdziwy "trending" workflow?** Wymagałby porównania dwóch okresów: np. 0–14 dni temu vs 15–29 dni temu. Frazy z rosnącym `impressions` między okresami = naprawdę trending. To wymaga dwóch getterów lub gettera z dwoma zakresami dat i node AI do porównania.

**Czy 20 tasków tygodniowo to problem?** Tak — zwłaszcza gdy taski są dla tych samych "top fraz" co tydzień. Lista zadań szybko się zaśmieca duplikatami. Lepszy wzorzec: jeden task zbiorczy z listą fraz zamiast 20 osobnych.

**Czy workflow ma wartość mimo błędnej nazwy?** Tak — przegląd top 20 fraz wyszukiwania co tydzień jest przydatny. To dobry punkt wyjścia do rozszerzenia słów kluczowych. Ale należy zmienić nazwę na "High Volume Search Terms" lub "Top Search Terms" żeby nie mylić użytkowników.

## Podsumowanie #10

| | |
|---|---|
| **Status** | ⚠️ działa technicznie, mylące koncepcyjnie |
| **Taski utworzone** | ~20 tasków per uruchomienie (redundantny filtr nie eliminuje niczego) |
| **Znane problemy** | 1. Nazwa "Trending" sugeruje wzrost — workflow pokazuje popularność bezwzględną. 2. Filtr `impressions>100` redundantny przy limicie TOP 20. 3. Task per fraza = 20 tasków/tydzień z dużą powtarzalnością. 4. Stary format triggera. |
| **Ocena ogólna** | Przydatny jako "weekly top searches review" — zła nazwa i zły wzorzec tworzenia tasków (20 per uruchomienie). Łatwy do naprawienia. |
| **Rekomendacja** | Zmienić nazwę na "High Volume Search Terms Monitor". Usunąć redundantny filtr lub zastąpić go sensowniejszym kryterium. Zmienić wzorzec: jeden zbiorczy task zamiast 20 osobnych. |

# Test #11 — `conversion_comparison`

**Data testu:** 2026-03-27

## ⚠️ UWAGA: To nie jest szablon — to prawdziwy workflow użytkownika

Plik zawiera:

- `creator.email: michal.strzebala@sembot.io` — email pracownika Sembot
- `creator.name: Michał Strzębała`
- `project_id: 909` — hardcoded w węzłach getter i create_task
- `connection_id: 652` — hardcoded w węzłach getter
- `configured: true`, `start_date: 2025-11-28`
- `timezone: Africa/Abidjan` — strefa czasowa Afryki Zachodniej (UTC+0) zamiast Europe/Warsaw

**Problem GDPR:** imię, nazwisko i email pracownika w pliku asset ładowanym przez wszystkich użytkowników. Musi zostać zastąpiony prawdziwym templatem.

---

## Węzeł 1 — Trigger

- 📋 `frequency: hourly`, `interval: 1` — co godzinę
- ❌ `timezone: Africa/Abidjan` — prawdopodobnie błąd konfiguracji
- ❌ Uruchamianie co godzinę dla porównania dwóch 30-dniowych okresów = zbędne obciążenie, dane się nie zmieniają tak szybko

---

## Węzeł 2 — Getter: `get_google_campaign`

- ❌ `connection_id: 652` — hardcoded
- ❌ `project_id: 909` — hardcoded
- 📋 `campaign_statuses: ["ENABLED"]`

**Dane:**

```
[
  { "campaignId": "201", "campaignName": "Brand - Exact" },
  { "campaignId": "202", "campaignName": "Non-Brand - Shoes" },
  { "campaignId": "203", "campaignName": "Shopping - All" }
]
```

---

## Węzeł 4 — flow_control: `iterator` (po kampaniach)

- 📋 `sourceId: 1802` (node 2 — getter kampanii)

---

## Węzeł 3 — Getter: `get_google_ads_metrics` (ostatnie 30 dni)

- ❌ `connection_id: 652` — hardcoded
- ❌ `project_id: 909` — hardcoded
- 🤖 `start_date: {NOW().subtractNumberOfDays(30)}` — 25.02.2026
- 🤖 `end_date: {NOW()}` — 27.03.2026
- 🤖 `resourceId: {4.campaignId}` — z iteratora

**Dane (kampania Brand - Exact, ostatnie 30 dni):**

```
{ "conversions": 45, "cost": 1240.50, "clicks": 312, "ctr": 0.058 }
```

---

## Węzeł 5 — Getter: `get_google_ads_metrics` (poprzednie 30 dni)

- 🤖 `start_date: {NOW().subtractNumberOfDays(60)}` — 26.01.2026
- 🤖 `end_date: {NOW().subtractNumberOfDays(30)}` — 25.02.2026

**Dane (kampania Brand - Exact, poprzednie 30 dni):**

```
{ "conversions": 58, "cost": 1180.20, "clicks": 298, "ctr": 0.054 }
```

---

## Węzeł 9 — flow_control: `filter`

- 📋 `percentageChange({3.conversions}, {5.conversions}) < {globals.conversion_decrease}`
- 👤 `globals.conversion_decrease` = -5

Obliczenie: `percentageChange(45, 58)` = `(45 - 58) / 58 × 100` = -22.4%

Warunek: `-22.4% < -5%` → ✅ PRZECHODZI (konwersje spadły o 22.4%, próg to -5%)

⚠️ **Uwaga na kierunek porównania:** `percentageChange(obecne, poprzednie)` — jeśli obecne < poprzednie, wynik jest ujemny. Warunek `< -5` wykrywa spadek ≥ 5%. Logika jest prawidłowa.

**Drugi warunek z description:** "koszt < 100 PLN" — **nie ma go w filtrze**. Tylko w description taska. Opis taska mówi "Warunki alertu: Spadek konwersji <30% i koszt <100 PLN" — ale faktyczny próg to -5%, nie -30%, i brak warunku na koszt.

---

## action: `create_task`

**Task — kampania Brand - Exact:**

> ***Tytuł:*** *`Kampania Brand - Exact: Spadek konwersji -22.4% i koszt 1240.5 PLN`*

>

> ***Opis:***

> *Kampania Google Ads: Brand - Exact (ID: 201)*

> *📊 Porównanie okresów:*

> *• Ostatnie 30 dni (25.02.2026 - 27.03.2026): Konwersje: 45 | Koszt: 1240.5 PLN | Kliknięcia: 312 | CTR: 5.8%*

> *• Poprzednie 30 dni (26.01.2026 - 25.02.2026): Konwersje: 58 | Koszt: 1180.2 PLN | Kliknięcia: 298 | CTR: 5.4%*

> *📈 Zmiana konwersji: -22.4%*

> *⚠️ Warunki alertu: Spadek konwersji <30% i koszt <100 PLN (nota: opis niezgodny z faktyczną logiką)*

---

## Pytania i wnioski z testu

**Czy logika porównania dwóch 30-dniowych okien jest dobra?** Tak — to najlepsza metoda detekcji trendów konwersji. Wyeliminowane są sezonowe tygodniowe wahania (7 vs 7 dni nie pokazuje trendu długoterminowego). 30 vs 30 dni to właśnie wzorzec stosowany przez profesjonalistów SEM.

**Dlaczego trigger co godzinę to problem?** Dane Google Ads aktualizują się z opóźnieniem 3–4 godzin. Porównanie 30-dniowych okresów nie zmienia się co godzinę w istotny sposób. Przy spadku konwersji alarm będzie generował taski co godzinę (brak deduplikacji) = task flood.

**Czy opis taska ma błąd?** Tak — treść `⚠️ Warunki alertu: Spadek konwersji <30% i koszt <100 PLN` jest błędna. Faktyczny próg to -5% i brak warunku na koszt.

---

## Podsumowanie #11

| | |
|---|---|
| **Status** | ❌ nie jest templatem — zawiera dane prawdziwego użytkownika |
| **Logika biznesowa** | ✅ wzorcowa — porównanie dwóch 30-dniowych okresów, konfigurowalny próg, bogaty opis taska |
| **Znane problemy** | 1. GDPR: imię, nazwisko i email pracownika w assets. 2. Hardcoded connection_id/project_id. 3. Trigger co 1h = task flood przy spadku konwersji. 4. Timezone: Africa/Abidjan (błąd konfiguracji). 5. Opis taska niezgodny z faktyczną logiką filtra. |
| **Ocena ogólna** | Logika biznesowa jest jedną z najlepszych w całym zestawie. Jako template bezużyteczny — wymaga przepisania. |
| **Rekomendacja** | Przepisać jako prawdziwy template: usunąć hardcoded ID, zmienić trigger na weekly, naprawić timezone, poprawić opis taska |

# Test #12 — `ai_weekly_report`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 1"` — stary format, poniedziałek 08:00

---

## Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id` — puste
- 👤 `project_id` — puste
- 📋 `resource: "campaign"`
- 🤖 `startDate`: ostatnie 7 dni (20.03.2026)
- 🤖 `endDate`: dziś (27.03.2026)

**Dane (zmyślone, format campaign performance):**

```
[
  { "campaignName": "Brand - Exact", "cost": 1240.50, "clicks": 312, "conversions": 45, "ctr": 0.058, "impressions
  { "campaignName": "Non-Brand - Shoes", "cost": 3450.20, "clicks": 876, "conversions": 28, "ctr": 0.032, "impress
  { "campaignName": "Shopping - All Products", "cost": 5120.00, "clicks": 2140, "conversions": 112, "ctr": 0.078,
  { "campaignName": "Remarketing - All Visitors", "cost": 680.30, "clicks": 890, "conversions": 18, "ctr": 0.12, "
  { "campaignName": "Display - Awareness", "cost": 420.00, "clicks": 145, "conversions": 2, "ctr": 0.009, "impress
]
```

---

## Węzeł 3 — ai_agent

- 👤 `agent_id: null` — użytkownik musi skonfigurować agenta AI

⚠️ **Problem z promptem:** Prompt odwołuje się do `{2.campaignName}`, `{2.cost}`, `{2.clicks}` itp. Ale node 2 zwraca **tablicę** kampanii, a nie pojedynczy obiekt. Bez iteratora, `{2.campaignName}` to wartość z pierwszego elementu tablicy — AI dostaje dane tylko jednej kampanii.

Jednak w praktyce — w zależności od implementacji silnika workflow — `{2.X}` w kontekście ai_agent może być interpolowane jako cała tablica lub jako pierwsze wystąpienie. To niejednoznaczne i ryzykowne.

**Symulacja inputu do AI (zakładamy że dostaje wszystkie dane):**

```
Analyze the following Google Ads campaign performance data for the past 7 days and provide:
1) Top performing campaigns, 2) Underperforming campaigns...
Data: Brand - Exact - Cost: 1240.5, Clicks: 312, Conversions: 45, CTR: 0.058, CPA: 1240.5/45
Non-Brand - Shoes - Cost: 3450.2, Clicks: 876, Conversions: 28, CTR: 0.032, CPA: 3450.2/28
Shopping - All Products - Cost: 5120, Clicks: 2140, Conversions: 112, CTR: 0.078, CPA: 5120/112
Remarketing - All Visitors - Cost: 680.3, Clicks: 890, Conversions: 18, CTR: 0.12, CPA: 680.3/18
Display - Awareness - Cost: 420, Clicks: 145, Conversions: 2, CTR: 0.009, CPA: 420/2
```

⚠️ **CPA hardcoded jako `{2.cost}/{2.conversions}`** — to tekst w prompcie, nie wyliczona wartość. AI dostaje literalnie "1240.5/45" i musi samodzielnie obliczyć. Powinno być `{divide({2.cost},{2.conversions})}` albo AI to oliicza samo.

**Symulowana odpowiedź AI:**

```
📊 Weekly Performance Report — 20.03–27.03.2026

✅ Top performing campaigns:
  • Shopping - All Products: 112 conversions, CPA: 45.71 PLN, CTR: 7.8% — excellent efficiency
  • Remarketing - All Visitors: highest CTR (12%), 18 conversions, CPA: 37.79 PLN — very efficient

⚠️ Underperforming campaigns:
  • Display - Awareness: CTR 0.9%, only 2 conversions from 16,120 impressions, CPA: 210 PLN — consider pausing or a
  • Non-Brand - Shoes: CTR 3.2%, CPA: 123.22 PLN — above target, review keywords and bids

💡 Budget recommendations:
  • Increase Shopping budget by 20% (strong ROAS, low CPA)
  • Reduce Display - Awareness spend (high cost, low conversion rate)
  • Review Non-Brand keyword quality scores

📈 Overall insights:
  • Total conversions: 205 | Total cost: 10,911 PLN | Avg CPA: 53.22 PLN
  • CTR above 5% in Brand and Shopping campaigns — healthy
  • Remarketing delivering best CTR — consider expanding audience lists
```

---

### Węzeł 4 — action: `create_task`

- 🤖 `name: "Weekly AI Performance Report - 2026-03-27"`
- 🤖 `description: {3.output}` — output z AI agenta
- ❌ Stary format (`action_type: create_task` w `data`)
- ❌ Brak `project_id` w payloadzie

**Task:**

> ***Tytuł:*** *`Weekly AI Performance Report - 2026-03-27`*

> ***Opis:*** *(odpowiedź AI jak powyżej)*

---

### Pytania i wnioski z testu

**Czy getter bez iteratora jest poprawny dla ai_agent?** To zależy od implementacji. Jeśli `{2.X}` przekazuje tylko pierwszy element tablicy — AI dostaje analizę jednej kampanii zamiast wszystkich. To jest ryzykowny wzorzec. Bezpieczniejszy byłby getter z `limit` + ai_agent który dostaje całą tablicę jako JSON.

**Czy 7 dni to dobry okres dla tygodniowego raportu?** Tak — raport tygodniowy logicznie obejmuje ostatnie 7 dni. ✅

**Czy AI jest w stanie prawidłowo przetworzyć takie dane?** Tak — to typowy use case dla LLM: analiza ustrukturyzowanych danych, identyfikacja outlierów, generowanie rekomendacji. Jakość zależy od modelu (agent_id) i od tego ile danych dostaje (problem tablicy).

**Ile tasków generuje tygodniowo?** 1 task per tydzień — zbiorczy raport. To właściwy wzorzec (w przeciwieństwie do #9/#10 które tworzą task per element). ✅

---

### Podsumowanie #12

| | |
|---|---|
| **Status** | ⚠️ działa, ale z ryzykiem niepełnych danych w prompcie |
| **Taski utworzone** | 1 per tydzień (zbiorczy raport) ✅ |
| **Znane problemy** | 1. `{2.X}` w prompcie może przekazywać tylko pierwszy element tablicy zamiast wszystkich kampanii. 2. CPA w prompcie jako wyrażenie tekstowe zamiast obliczonej wartości. 3. `agent_id: null` — wymaga konfiguracji. 4. Stary format triggera i create_task. 5. Brak `project_id` w create_task. |
| **Ocena ogólna** | Dobry pomysł i właściwy wzorzec (1 zbiorczy task). Problem z przekazywaniem tablicy do AI promptu wymaga weryfikacji zachowania systemu. |
| **Rekomendacja** | Zweryfikować jak `{2.X}` działa w ai_agent z getterem zwracającym tablicę. Rozważyć dodanie `limit` do gettera. Przenieść na nowy format. |

## Test #13 — `campaign_intelligence_scoring`

**Data testu:** 2026-03-27

### Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 1"` — stary format, poniedziałek 08:00

---

### Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id`, `project_id` — puste
- 📋 `resource: "campaign"`, 🤖 30-dniowe okno

Identyczna konfiguracja jak w #12 (`ai_weekly_report`) — ta sama struktura danych wejściowych.

---

### Węzeł 3 — ai_agent

- 👤 `agent_id: null`

Prompt pyta o ocenę kampanii w skali 1–10 z mocnymi/słabymi stronami i priorytetowym działaniem. Ten sam problem co w #12: `{2.campaignName}` w prompcie może przekazywać tylko pierwszy element tablicy.

Dodatkowo w prompcie: `ROAS: {2.conversions_value}/{2.cost}` — wyrażenie tekstowe zamiast obliczonej wartości. AI musi samodzielnie podzielić.

**Symulowana odpowiedź AI:**

```
📊 Campaign Intelligence Scoring — 27.03.2026

• Shopping - All Products — Score: 9/10

  ✅ Mocne: najwyższe konwersje (112), najniższe CPA (45.71), wysoki CTR
  ⚠️ Słabe: wysoki budżet, warto monitorować ROAS
  🎯 Akcja: zwiększyć budżet o 15-20%

• Remarketing - All Visitors — Score: 8/10

  ✅ Mocne: najwyższy CTR (12%), dobry CPA (37.79)
  ⚠️ Słabe: mała skala (18 konwersji)
  🎯 Akcja: rozszerzyć listy remarketingowe

• Brand - Exact — Score: 7/10

  ✅ Mocne: wysokie konwersje relatywnie do kosztu
  ⚠️ Słabe: ograniczone słowami brandowymi, brak zasięgu na nowych użytkowników
  🎯 Akcja: utrzymać, sprawdzić search impression share

• Non-Brand - Shoes — Score: 5/10

  ⚠️ Słabe: CPA 123.22 PLN, niski CTR (3.2%)
  🎯 Akcja: przegląd słów kluczowych, obniżyć stawki lub wstrzymać słowa droższe od progu

• Display - Awareness — Score: 3/10

  ❌ CTR 0.9%, CPA 210 PLN, 2 konwersje z 16k wyświetleń
  🎯 Akcja: wstrzymać lub zmienić targetowanie
```

---

### Węzeł 4 — action: `create_task`

- 🤖 `name: "Campaign Intelligence Report - 2026-03-27"`
- ❌ Stary format, brak `project_id`

**Task:**

> ***Tytuł:*** *`Campaign Intelligence Report - 2026-03-27`*

> ***Opis:*** *(scoringowy raport AI jak powyżej)*

---

## Różnica vs #12 (`ai_weekly_report`)

| Cecha | #12 ai_weekly_report | #13 campaign_intelligence_scoring |
|---|---|---|
| Okres danych | 7 dni | 30 dni |
| Output AI | analiza + rekomendacje | scoring 1–10 + mocne/słabe strony |
| Fokus | "co się stało w tym tygodniu" | "jak kamapanie stoją w skali miesiąca" |
| Wartość dla użytkownika | operacyjna (tygodniowa) | strategiczna (miesięczna) |

Oba są wartościowe — uzupełniają się nawzajem.

---

## Podsumowanie #13

| | |
|---|---|
| **Status** | ⚠️ działa, ale z tymi samymi ryzykami co #12 |
| **Taski utworzone** | 1 per tydzień (zbiorczy) ✅ |
| **Znane problemy** | 1. `{2.X}` w prompcie — ryzyko tylko pierwszego elementu tablicy. 2. `agent_id: null`. 3. ROAS i CPA jako wyrażenia tekstowe. 4. Stary format. |
| **Ocena ogólna** | Dobry szablon strategiczny — scoring kampanii to wysokowartościowy use case. Wymaga weryfikacji przekazywania tablicy do AI. |
| **Rekomendacja** | Naprawić problem tablicy/promptu. Dodać `globals` dla parametrów scoringu (progi CPA/ROAS). Przenieść na nowy format. |

---

# Test #14 — `pmax_analysis`

**Data testu:** 2026-03-27

## ⚠️ UWAGA: Trzy warstwy problemów w jednym szablonie

---

### Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 1"` — stary format

---

### Węzeł 2 — Getter: `get_google_merchant_center_performance_info`

- 👤 `connection_id`, `project_id` — puste
- 🤖 30-dniowe okno

**Rzeczywiste pola tego gettera:**

```
title (kategoria/brand), clicks, impressions, ctr, conversions, conversionValue
```

Dane są po brand/kategorii — NIE po konkretnym produkcie. To agregaty na poziomie brand.

---

### Węzeł 3 — Getter: `get_google_merchant_center_top_products`

- 👤 `connection_id`, `project_id` — puste

**Rzeczywiste pola `BestSellersProductClusterViewDTO`:**

```
productClusterTitle, productClusterBrand, productClusterInventoryStatus,
bestSellersRank, bestSellersRelativeDemandChange, reportCategoryId
```

**Pola których NIE MA** (ale których szuka prompt):

- `{3.title}` ❌ — jest `productClusterTitle`, ale referencja `{3.title}` nie zadziała
- `{3.price}` ❌ — pole nie istnieje w tym DTO
- `{3.availability}` ❌ — jest `productClusterInventoryStatus`, ale nie `availability`
- `{3.conversions}` ❌ — pole nie istnieje (to market-level data, nie merchant)

---

### Węzeł 4 — ai_agent

- 👤 `agent_id: null`

**Problem strukturalny:** oba gettery (node 2 i node 3) mają `parent_id: null` — są niezależne od siebie i oba biegną równolegle od triggera. `ai_agent` też ma `parent_id: null`, a nie `parent_id: 2` lub `parent_id: 3`. Zależności między węzłami nie są poprawnie ustawione.

W prompcie: `{3.title}`, `{3.price}`, `{3.availability}`, `{3.conversions}` — żadne z tych pól nie istnieje w `BestSellersProductClusterViewDTO`. AI dostanie puste/null wartości dla datasourcera node 3.

**Fundamentalny błąd koncepcyjny:** `get_google_merchant_center_top_products` zwraca **bestsellery całego Google Shopping** (market-level), nie produkty tego merchantów. AI analizuje rynek, a nie konto użytkownika. Użytkownik myśli że analizuje swoje produkty.

**Symulacja inputu AI (z realnych danych, ignorując błędy pól):**

```
Analyze Shopping/PMax performance.
GMC Performance: Nike - Clicks: 2340, Impressions: 45600, Conversions: 89
Top Products: [null] - Price: [null], Availability: [null], Conversions: [null]
```

AI dostaje połowę danych i połowę wartości null. Wygeneruje analizę ale na bazie niepełnych danych, bez ostrzeżenia użytkownika że dane są niekompletne.

## Węzeł 5 — action: `create_task`

- 🤖 `name: "Shopping/PMax Analysis - 2026-03-27"`
- ❌ Stary format, brak `project_id`

---

## Pytania i wnioski z testu

**Dlaczego nazwa "Performance Max" jest myląca?** Getter `get_google_merchant_center_performance_info` zwraca dane GMC per brand/kategoria — nie rozróżnia kampanii PMax od Shopping. Nie ma tu żadnych danych specyficznych dla PMax (asset groups, signals, etc.). To po prostu raport GMC.

**Co byłoby prawidłowym uzupełnieniem danych PMax?** Google Ads API ma `get_google_ads_performance resource:campaign` — po filtrowaniu po `campaignType: PERFORMANCE_MAX` można by uzyskać dane kampanii PMax. Ten getter jest dostępny, ale nie użyty.

---

## Podsumowanie #14

| | |
|---|---|
| **Status** | ❌ potrójny błąd: zły getter market-level, nieistniejące pola, błędna struktura węzłów |
| **Taski utworzone** | 1 (ale z niepełnymi/błędnymi danymi w prompcie) |
| **Znane problemy** | 1. `top_products` → market-level data, nie produkty merchantów. 2. Nieistniejące pola: `{3.title}`, `{3.price}`, `{3.availability}`, `{3.conversions}`. 3. Węzły getter z `parent_id: null` — brak jawnej zależności między getterami a ai_agent. 4. Nazwa "PMax" myląca — brak danych PMax-specific. 5. Stary format. |
| **Ocena ogólna** | Jeden z gorzej zaprojektowanych szablonów — wiele warstw błędów naraz. Idea jest dobra (cross-data AI analysis), realizacja wymaga przebudowy. |
| **Rekomendacja** | Przepisać lub usunąć. Wariant naprawiony: `get_google_ads_performance resource:campaign` (filtr PMax) + `get_google_merchant_center_price_competitiveness_info` (merchant-specific). |

---

# Test #15 — `seasonal_ad_copy_check`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 1 "` — stary format, 1. każdego miesiąca 08:00

---

## Węzeł 2 — Getter: `get_google_ads_ad_copy`

- 👤 `connection_id`, `project_id` — puste

**Dane (zmyślone, format AdCopyDTO):**

```
[
  { "campaignName": "Brand", "adGroupName": "Nike", "adStatus": "ENABLED", "headline1": "Nike Air Max – Sale", "he
  { "campaignName": "Non-Brand", "adGroupName": "Sneakers", "adStatus": "PAUSED", "headline1": "Buty Sportowe Onli
  { "campaignName": "Shopping", "adGroupName": "All Products", "adStatus": "ENABLED", "headline1": "Kupuj Teraz",
]
```

---

## Węzeł 3 — flow_control: `filter`

- 📋 `adStatus = "ENABLED"`

Wynik: reklama "Non-Brand/Sneakers" (PAUSED) odpada. Przechodzą 2 reklamy: Brand/Nike i Shopping/All Products.

---

## Węzeł 4 — flow_control: `iterator`

2 przebiegi (po jednej reklamie).

---

## Węzeł 5 — ai_agent (per reklama)

- 👤 `agent_id: null`

Prompt dostaje konkretną treść reklamy — nagłówki, opisy, URL. AI ocenia sezonowość, zgodność, jakość.

**Symulacja AI dla reklamy Brand/Nike:**

```
📝 Ad Copy Review – Brand / Nike | 27.03.2026

1) Outdated seasonal references:
   ❌ "Letnia Wyprzedaż -30%" w headline3 – marzec 2026, nie ma lata. Ten nagłówek nie jest aktualny.
   ❌ "Sprawdź letnie nowości Nike" w description1 – ten sam problem.

2) Headline quality:
   ✅ "Nike Air Max – Sale" – mocny, brandowy, z CTA
   ✅ "Darmowa Dostawa" – dobry benefit
   ⚠️ "Letnia Wyprzedaż -30%" – usuń lub zamień na "Nowości Wiosna 2026" lub "Bestsellery"

3) Compliance:
   ✅ Brak oczywistych naruszeń polityki Google Ads

4) Sugestie:
   - Zamień "Letnia Wyprzedaż -30%" → "Najnowsze Modele 2026" lub "Bestsellery Nike"
   - Zmień "Sprawdź letnie nowości" → "Odkryj nowości wiosna-lato 2026"
```

---

## Węzeł 6 — action: `create_task`

- 🤖 `name: "Ad copy review: Brand / Nike"`

- ❌ Stary format, brak `project_id`

**Task #1:**

> ***Tytuł:*** `Ad copy review: Brand / Nike`

> ***Opis:*** *(analiza AI jak powyżej)*

**Task #2:**

> ***Tytuł:*** `Ad copy review: Shopping / All Products`

> ***Opis:*** *(analogiczna analiza)*

---

## Pytania i wnioski z testu

**Czy wzorzec "task per reklama" jest problematyczny?** Tak — konto z 200 ENABLED reklamami = 200 tasków miesięcznie z jednego workflow. Lista zadań może być zaśmiecona. Lepszy wzorzec: grupować po kampanii (wszystkie reklamy kampanii → AI → 1 task per kampania).

**Czy AI faktycznie widzi treść reklam?** Tak — getter `get_google_ads_ad_copy` zwraca headline1–3, description1–2, URL. AI ma dostęp do realnej treści. To odróżnia ten szablon od `ms_seasonal_ad_copy_check` który AI nie widzi żadnej reklamy. ✅

**Czy "compliance check" jest realny?** Częściowo — AI może wykryćoczywiste naruszenia polityki (misleading claims, forbidden words) ale nie ma dostępu do aktualnej polityki Google Ads i nie zastępuje oficjalnego procesu review.

---

## Podsumowanie #15

| | |
|---|---|
| **Status** | ✅ działa end-to-end — AI faktycznie widzi treść reklam |
| **Taski utworzone** | 2 (po jednym per ENABLED reklama) |
| **Znane problemy** | 1. Task per reklama = problem skali (200 reklam = 200 tasków/miesiąc). 2. Stary format triggera. 3. Stary format create_task (brak project_id). |
| **Ocena ogólna** | Jeden z wartościowszych szablonów AI — realny dostęp do treści reklam, sensowne pytania w prompcie. Problem skalowalności przy dużych kontach. |
| **Rekomendacja** | Zmienić wzorzec na "task per kampania" zamiast "task per reklama". Dodać limit w getterze lub grupowanie po `campaignName`. |

---

# Test #16 — `daily_ppc_brief`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: daily`, `time: "08:00"` — nowy format ✅
- 👤 `timezone` — domyślnie Europe/Warsaw

---

## Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id: null`, `project_id: null` — do uzupełnienia
- 📋 `resource: "campaign"`, 🤖 ostatnie 7 dni

---

## Węzeł 3 — flow_control: `filter`

- 📋 `cost > 0` — tylko kampanie z wydatkami w ostatnich 7 dniach

✅ Dobry filtr — eliminuje kampanie PAUSED lub bez ruchu.

---

## Węzeł 4 — ai_agent

- 👤 `agent_id: null`

**Uwaga na referencje:** Prompt używa `{2.campaignName}`, `{2.cost}` itp. — to node 2 (getter), a nie node 3 (po filtrze). Dane które AI dostaje mogą zawierać kampanie z cost=0 które filtr odrzucił. Prawidłowa referencja powinna być `{3.X}` (po filtrze).

**Dane wejściowe AI (po filtrze, zakładamy że prompt dostaje node 3 mimo błędnej referencji):**

```
Write a concise daily PPC brief for today, Friday, March 27, 2026.
Based on Google Ads data from the last 7 days:
Brand - Exact — Cost: 1240.5, Clicks: 312, Conversions: 45, CTR: 5.8%, ROAS: 3.2
Non-Brand - Shoes — Cost: 3450.2, Clicks: 876, Conversions: 28, CTR: 3.2%, ROAS: 1.8
Shopping - All Products — Cost: 5120, Clicks: 2140, Conversions: 112, CTR: 7.8%, ROAS: 4.1
Remarketing - All Visitors — Cost: 680.3, Clicks: 890, Conversions: 18, CTR: 12%, ROAS: 5.2
```

**Symulowana odpowiedź AI (max 150 słów):**

```
📊 Account health: Strong week overall — 203 conversions across 4 campaigns at avg CPA 54 PLN.

✅ Star of the week: Shopping - All Products delivering 112 conversions with ROAS 4.1 —
healthy efficiency, consider increasing budget before weekend.

🔍 Check today: Non-Brand - Shoes shows ROAS 1.8, below typical target.
Review search terms for irrelevant queries draining budget. Check if any keywords
hit daily budget cap.

🎯 Action before EOD: Pause 3–5 lowest-performing Non-Brand keywords
(filter by cost>15 PLN, conversions=0 last 7d). Quick win, no creative needed.
```

---

## Węzeł 5 — action: `create_task`

- 🤖 `title: "☀️ Daily PPC Brief — 27.03.2026"`
- 👤 `project_id: null`
- ✅ Nowy format create_task

**Task:**

> ***Tytuł:*** *`☀️ Daily PPC Brief — 27.03.2026`*

> ***Opis:*** *(brief AI 150 słów jak powyżej)*

---

## Pytania i wnioski z testu

**Czy 365 tasków rocznie to problem?** Tak — jeśli taski nigdy nie są zamykane automatycznie, lista zadań w ciągu roku będzie zawierać 365 briefów. Wymaga dyscypliny użytkownika w zamykaniu tasków.

**Błąd referencji `{2.X}` vs `{3.X}`:** Prompt odwołuje się do `{2.campaignName}` (raw getter) zamiast `{3.campaignName}` (po filtrze cost>0). W praktyce może to nie robić różnicy jeśli wszystkie kampanie mają cost>0, ale jest to niejednoznaczne i może powodować problemy przy kampaniach z zerowym kosztem.

**Prompt jest wzorcowy:** "150 words max", "no fluff", "one concrete action" — to dokładnie to czego potrzeba w daily brief. Jeśli AI dobrze zrozumie instrukcję, output będzie actionable. ✅

---

## Podsumowanie #16

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 1 dziennie (zbiorczy brief) ✅ |
| **Znane problemy** | 1. Prompt używa `{2.X}` zamiast `{3.X}` po filtrze — potencjalnie niefiltrowane dane w prompcie. 2. 365 tasków/rok bez auto-close. 3. `agent_id: null`. |
| **Ocena ogólna** | Jeden z lepiej zaprojektowanych szablonów — nowy format, zbiorczy task, właściwy prompt z ograniczeniem długości. Drobny błąd referencji do naprawienia. |
| **Rekomendacja** | Poprawić `{2.X}` → `{3.X}` w prompcie. Rozważyć dodanie `due_date` na +1 dzień żeby brief był automatycznie "widoczny" przez 1 dzień. |

---

# Test #17 — `weekly_opportunity_scout`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`, `days_of_week: ["monday"]`, `time: "08:00"` — nowy format ✅

---

## Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id: null`, `project_id: null`
- 📋 `resource: "search_term_view"`, 🤖 ostatnie 7 dni
- 📋 `sortField: "impressions"`, `sortOrder: "DESC"`, `limit: 100`

✅ Limit 100 to rozsądna wartość — wystarczająco dużo danych dla AI bez przeciążenia.

**Dane (zmyślone, top search terms per impressions):**

```
[
  { "title": "nike air max 90 damskie", "campaign_name": "Non-Brand", "impressions": 890, "clicks": 78, "conversio
  { "title": "adidas running", "campaign_name": "Non-Brand", "impressions": 750, "clicks": 45, "conversions": 1, "
  { "title": "buty do biegania tanie", "campaign_name": "Non-Brand", "impressions": 620, "clicks": 38, "conversion
  { "title": "sneakers na lato 2026", "campaign_name": "Non-Brand", "impressions": 410, "clicks": 28, "conversions
  { "title": "białe sneakersy damskie", "campaign_name": "Non-Brand", "impressions": 380, "clicks": 22, "conversio
]
```

---

## Węzeł 3 — flow_control: `filter`

- 👤 `globals.min_impressions` = 50 (domyślnie)

Przy danych powyżej: wszystkie 5 fraz ma > 50 wyświetleń → wszystkie przechodzą.

⚠️ Podobnie jak w #17 (`trending_search_terms`) — filtr jest redundantny przy `limit: 100 sortBy: impressions DESC`. Przy typowym koncie żadna z top 100 fraz nie będzie miała < 50 wyświetleń.

**Uwaga na referencję w prompcie:** Prompt używa `{2.title}`, `{2.campaign_name}` — node 2 (getter przed filtrem), nie node 3 (po filtrze). Ten sam problem co w #16.

---

## Węzeł 4 — ai_agent

**Prompt jest wzorcowy:**

- Jasna instrukcja: grupowanie po temacie/intencji
- Konkretny output: sugeriowane słowa kluczowe + match type + priorytet
- Sensowny warunek: "2+ powiązane frazy"
- Fokus: pomijaj frazy z 0 konwersji i niskim CTR

**Symulowana odpowiedź AI:**

```
🔍 Weekly Opportunity Scout — 27.03.2026

CLUSTER 1: Nike / Branded — High Priority ⭐
  • "nike air max 90 damskie" (890 imp, 4 conv)

→ Dodaj keyword: [nike air max 90 damskie] — exact match
→ Rozważ osobną ad group: "Nike Air Max Women"

CLUSTER 2: Running / Performance
  • "adidas running" (750 imp, 1 conv)

  • "buty do biegania tanie" (620 imp, 3 conv)

→ Dodaj: [buty do biegania] phrase match — medium priority
→ Dodaj: +tanie +buty +bieganie — broad modified (jeśli dostępny)

CLUSTER 3: Seasonal / Summer 2026
  • "sneakers na lato 2026" (410 imp, 2 conv)

→ Dodaj: [sneakers lato 2026] — exact match — high priority (sezonowa fraza)

POMINIĘTE:
  • "białe sneakersy damskie" (0 conv, CTR 5.8%) — za mało danych konwersji, obserwować
```

---

### action: `create_task`

- 🤖 `title: "🔍 Weekly Opportunity Scout — 27.03.2026"`
- 👤 `project_id: null`
- ✅ Nowy format create_task

---

### Podsumowanie #17

| | |
|---|---|
| **Status** | ✅ działa end-to-end — jeden z najlepszych szablonów |
| **Taski utworzone** | 1 per tydzień (zbiorczy) ✅ |
| **Znane problemy** | 1. Filtr `{globals.min_impressions}` redundantny przy `limit:100 sortBy:impressions`. 2. Prompt używa `{2.X}` zamiast `{3.X}` po filtrze. 3. `agent_id: null`. |
| **Ocena ogólna** | Jeden z najwartościowszych szablonów AI — actionable output (konkretne słowa do dodania), właściwy wzorzec (1 task/tydzień), dobry prompt z jasnymi instrukcjami i sensownym limitem danych. |
| **Rekomendacja** | Poprawić referencję `{2.X}` → `{3.X}`. Filtr zostawić (może być przydatny dla małych kont). Wzorzec do naśladowania przy tworzeniu MS Ads wariantów. |

---

## Test #18 — `search_terms_clustering`

**Data testu:** 2026-03-27

### Węzeł 1 — Trigger

- 📋 `frequency: monthly`, `days_of_month: "first_day"`, `time: "08:00"` — nowy format ✅

---

### Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id: null`, `project_id: null`
- 📋 `resource: "search_term_view"`, 🤖 ostatnie 30 dni
- 📋 `limit: 200` — większy limit niż #17 (200 vs 100), sensowne dla analizy miesięcznej

---

### Węzeł 3 — flow_control: `filter`

- 👤 `globals.min_impressions` = 30

---

### Węzeł 4 — ai_agent

**Prompt jest szczegółowy i precyzyjny:**

- Grupowanie po tematyce, typie produktu, intencji (informacyjna/komercyjna/transakcyjna)
- Konkretne kryteria tworzenia nowej ad group: 5+ fraz LUB 300+ łącznych wyświetleń
- Flagowanie klastrów sugerujących nowy typ kampanii (brand, competitor, kategoria)
- Frazy nie pasujące do klastrów = luki w strukturze

Prompt jest lepszy niż w #17 — bardziej precyzyjne kryteria, dodatkowy insight o lukach.

**Symulowana odpowiedź AI (skrócona):**

```
🗂️ Search Terms Clustering — February 2026

CLUSTER A: Nike Brand (8 fraz, 4,200 łącznych imp) → osobna ad group ✅
  • "nike air max 90", "nike air max damskie", "nike running", ...

→ Rekomendacja: utwórz "Nike - Branded" ad group, exact + phrase match

CLUSTER B: Running/Performance (12 fraz, 6,800 imp) → osobna ad group ✅
  • "buty do biegania", "buty sportowe bieganie", "running shoes tanie", ...

→ Rekomendacja: podziel na "Running - Tanie" (price-focused) i "Running - Premium"

CLUSTER C: Summer 2026 (3 frazy, 890 imp) → jeszcze za mały, obserwować
  • "sneakers lato 2026", "buty letnie 2026", "modne sneakersy wiosna"


LUKI STRUKTURALNE:
  • "buty dla dzieci" (450 imp, 2 conv) — brak kampanii dziecięcej, sprawdzić czy oferujemy

  • "buty do piłki nożnej" (380 imp, 0 conv) — poza ofertą? Dodać jako negative keyword
```

---

### action: `create_task`

- 🤖 `title: "🗂️ Search Terms Clustering — February 2026"`
- 👤 `project_id: null`

- ✅ Nowy format

---

## Różnica vs #17 (`weekly_opportunity_scout`)

| Cecha | #17 Opportunity Scout | #18 Clustering |
|---|---|---|
| Częstotliwość | Co tydzień | Co miesiąc |
| Limit danych | 100 fraz / 7 dni | 200 fraz / 30 dni |
| Output AI | "Dodaj te słowa kluczowe" | "Przestrukturyzuj te ad groups" |
| Horyzont działania | Krótkoterminowy (tydzień) | Strategiczny (miesiąc) |

Oba szablony są komplementarne i wartościowe. ✅

---

## Podsumowanie #18

| | |
|---|---|
| **Status** | ✅ działa end-to-end — wzorcowy szablon AI |
| **Taski utworzone** | 1 per miesiąc (zbiorczy) ✅ |
| **Znane problemy** | 1. `{2.X}` zamiast `{3.X}` w prompcie. 2. `agent_id: null`. |
| **Ocena ogólna** | Jeden z najlepiej zaprojektowanych szablonów AI — precyzyjny prompt, strategiczny output, właściwa częstotliwość, dobry wzorzec (1 task/miesiąc). |
| **Rekomendacja** | Poprawić referencję w prompcie. Wzorzec do naśladowania. |

---

# Test #19 — `monthly_cross_platform_summary`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: monthly`, `days_of_month: "first_day"`, `time: "08:00"` — nowy format ✅

---

## Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id: null`, `project_id: null`
- 📋 `resource: "campaign"`, 🤖 ostatnie 30 dni
- ⚠️ Brak `limit` — przy dużych kontach zwróci wszystkie kampanie

---

## Węzeł 3 — Getter: `get_microsoft_advertising_campaigns`

- 👤 `connection_id: null`, `project_id: null`
- 🤖 Ostatnie 30 dni
- 📋 `parent_id: 2` — uruchamiany po node 2 (Google Ads getter)

✅ Wzorzec sekwencyjny — MS Ads getter uruchamiany po Google Ads. Oba zestawy danych dostępne dla AI.

---

## Węzeł 4 — ai_agent

- 👤 `agent_id: null`

**Prompt jest bardzo dobry:**

- Jasna struktura outputu: 5 sekcji z konkretnymi wymaganiami
- Oba datasource w prompcie: `{2.X}` (Google) i `{3.X}` (Microsoft)
- Actionable finale: "3 konkretne akcje na następny miesiąc, rankowane po oczekiwanym wpływie"
- Instrukcja stylu: "Be direct. Use numbers. Skip vague observations."

⚠️ Ten sam problem co wcześniej: `{2.X}` i `{3.X}` mogą przekazywać tylko pierwszy element tablicy jeśli silnik nie obsługuje tablicowej interpolacji w promptach.

**Dane testowe:**

Google Ads (ostatnie 30 dni):

```
Brand - Exact: Cost 4820 PLN, Clicks 1280, Conv 178, CTR 5.8%, ROAS 4.2
Non-Brand: Cost 12340 PLN, Clicks 3240, Conv 89, CTR 3.1%, ROAS 1.9
Shopping: Cost 18900 PLN, Clicks 7820, Conv 412, CTR 7.2%, ROAS 5.1
```

Microsoft Ads (ostatnie 30 dni):

```
Brand - Bing: Spend 890 PLN, Clicks 234, Conv 28, CTR 4.1%, ROAS 3.8
Non-Brand - Bing: Spend 2340 PLN, Clicks 680, Conv 19, CTR 2.9%, ROAS 1.6
```

**Symulowana odpowiedź AI:**

```
📊 Monthly Summary — February 2026

Overall: Strong month driven by Google Shopping (ROAS 5.1, 412 conv).
Microsoft contributes 47 conversions at higher avg CPA — niche but profitable for brand terms.
Total: 706 conversions, ~39,290 PLN spend, avg ROAS 3.8.

Google Ads:
✅ Top: Shopping (ROAS 5.1, 412 conv) — scale this
❌ Problem: Non-Brand (ROAS 1.9, CPA 138 PLN) — keyword audit needed
💡 Insight: Brand captures intent cheaply (CPA ~27 PLN) — protect with bid modifier

Microsoft Ads:
✅ Top: Brand - Bing (ROAS 3.8, 28 conv) — similar efficiency to Google Brand
❌ Problem: Non-Brand - Bing (ROAS 1.6) — below target, review bid strategy
💡 Insight: Bing users convert from Brand terms at high rate — worth keeping

Cross-platform budget shift: Move 15% of Non-Brand - Bing budget → Google Shopping

Next month — Top 3 actions:
• Add 20% to Google Shopping budget (highest ROAS, most conversions) — expected +80 conv
• Pause bottom 10 Non-Brand Google keywords (cost>100 PLN, 0 conv last 30d) — save ~1200 PLN
• Test Remarketing on Bing (currently 0 budget) — replicate Google Remarketing success
```

---

### action: `create_task`

- 🤖 `title: "📊 Monthly Summary — February 2026"`
- 👤 `project_id: null`
- ✅ Nowy format

---

## Pytania i wnioski z testu

**Czy sekwencyjne gettery (node 2 → node 3) są prawidłową architekturą?** Tak — node 3 (`parent_id: 2`) uruchamia się po node 2. Ale oba gettery są niezależne (każdy pobiera swoje dane), sekwencja jest tylko techniczna. AI w node 4 (`parent_id: 3`) ma dostęp do obu datasource przez `{2.X}` i `{3.X}`. ✅

**Czy brak `limit` na getterze Google jest problemem?** Tak — przy koncie z 50+ kampaniami AI dostanie bardzo długi prompt. Optymalnie dodać `limit: 20` lub `sortBy: cost DESC limit: 20` żeby AI analizowało najbardziej znaczące kampanie.

---

## Podsumowanie #19

| | |
|---|---|
| **Status** | ✅ działa end-to-end — jeden z najlepszych szablonów |
| **Taski utworzone** | 1 per miesiąc (zbiorczy) ✅ |
| **Znane problemy** | 1. Brak `limit` w Google Ads getter — ryzyko przeciążenia promptu. 2. `{2.X}` / `{3.X}` w prompcie — ryzyko tylko pierwszego elementu. 3. `agent_id: null`. |
| **Ocena ogólna** | Wzorcowy cross-platform szablon — dual getter, strukturowany prompt z jasnymi sekcjami i actionable finałem. Jeden z najlepszych promptów w całym zestawie. |
| **Rekomendacja** | Dodać `sortField: "cost", sortOrder: "DESC", limit: 20` do obu getterów. Naprawić ryzyko tablicowej interpolacji. |

---

# Test #20 — `keyword_quality_audit`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`, `days_of_week: ["monday"]` — nowy format ✅

---

## Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id: null`, `project_id: null`
- 📋 `resource: "keyword_view"`, 🤖 ostatnie 30 dni
- ⚠️ Brak `limit` — przy dużych kontach setki słów kluczowych

---

## Węzeł 3 — flow_control: `filter`

- 👤 `globals.min_cost` = 10 (domyślnie)

Filtr `cost > 10` eliminuje słowa kluczowe z małymi wydatkami. Sensowny filtr — zmniejsza szum. ✅

---

## Węzeł 4 — ai_agent

**Prompt jest mocny i precyzyjny:**

- Konkretne kryteria oceny: `cost > 3x avg CPA AND conversions = 0`, `CTR < 0.5%`
- Dwie sekcje outputu: PAUSE CANDIDATES + WATCH LIST
- Instrukcja: "Use the actual keyword text, not paraphrases" — dobra praktyka

⚠️ Ten sam problem: `{2.title}` zamiast `{3.title}` po filtrze.

**Symulacja AI:**

```
PAUSE CANDIDATES:
  • "buty ortopedyczne tanie" — Campaign: Non-Brand, Cost: 87 PLN, 0 conv, CTR 0.3%

    → Cost > 3x avg CPA (29 PLN), CTR below 0.5%, likely out-of-scope intent. PAUSE.
  • "sportowe buty dziecięce" — Cost: 42 PLN, 0 conv, CTR 0.8%

    → Likely outside product catalog. PAUSE and verify against inventory.

WATCH LIST:
  • "adidas damskie zima" — Cost: 35 PLN, 1 conv, CTR 2.1%

    → Single conversion could be anomaly. Monitor this week before acting.
  • "białe sneakersy 2026" — Cost: 22 PLN, 0 conv, CTR 4.2%

    → Good CTR suggests intent match. Zero conv may be landing page issue. Check LP.
```

---

## action: `create_task`

- 🤖 `title: "🔑 Keyword Quality Audit — 27.03.2026"`
- 👤 `project_id: null`

---

## Podsumowanie #20

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 1 per tydzień (zbiorczy) ✅ |
| **Znane problemy** | 1. Brak `limit` w getter — ryzyko przeciążenia promptu AI. 2. `{2.X}` zamiast `{3.X}`. 3. `agent_id: null`. |
| **Ocena ogólna** | Bardzo dobry szablon audytowy — precyzyjny prompt z konkretnymi kryteriami, właściwe sekcje (PAUSE/WATCH). Konfigurowalny próg min_cost. |
| **Rekomendacja** | Dodać `limit: 50` do gettera. Naprawić referencję. |

# Test #21 — `master_account_audit`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`, `days_of_week: ["monday"]`, `time: "07:00"` — nowy format ✅

---

## Węzeł 2 — Getter: `get_google_ads_performance` (kampanie)

- 👤 `connection_id: null`, `project_id: null`
- 📋 `resource: "campaign"`, 🤖 ostatnie 7 dni
- ⚠️ Brak `limit`

---

## Węzeł 3 — Getter: `get_google_ads_performance` (search terms)

- 📋 `resource: "search_term_view"`, `limit: 50`, `sortField: "impressions"` ✅
- 📋 `parent_id: 2` — uruchamiany sekwencyjnie po node 2

✅ Dual-getter wzorzec — ten sam co #19, ale dla Google Ads.

---

## Węzeł 4 — ai_agent

**Prompt jest najlepszym w całym zestawie:**

- "Exactly 3 findings, ranked by urgency" — eliminuje padding i ogólniki
- "For each finding: problem in one sentence, specific numbers, one concrete action this week"
- "Look at campaigns AND search terms together" — cross-data analysis
- "No padding, no general advice" — jasna instrukcja stylu

**Dane:**

- Kampanie: 5 (Brand, Non-Brand, Shopping, Remarketing, Display)
- Search terms: top 50 po wyświetleniach

**Symulowana odpowiedź AI:**

```
Weekly Account Audit — 20.03 – 27.03.2026


FINDING #1 (URGENT): Non-Brand kampania traci 1200 PLN/tydzień bez proporcjonalnego zwrotu
Data: Non-Brand Cost 3450 PLN, Conv 28, CPA 123 PLN. Search terms ujawniają:
"buty do skoków wzwyż" (890 imp, 0 conv) i "buty baletowe" (640 imp, 0 conv) —
irrelevant queries draining budget.
Akcja: Dodaj "skoki wzwyż" i "baletowe" jako exact negative keywords w Non-Brand.
Szacowany oszczędność: 150-200 PLN/tydzień.


FINDING #2 (HIGH): Display - Awareness ma CTR 0.9% ale search terms wskazują
że użytkownicy szukają produktów których nie mamy w ofercie.
Data: "buty ortopedyczne dla dzieci" (420 imp, 0 conv) w kampanii Display.
Akcja: Wyklucz kategorie dziecięce z Display targeting jeśli nie oferujemy produktów dziecięcych.


FINDING #3 (MEDIUM): Shopping kampania nie pojawia się na top branded queries.
Data: Search term "nike air max original" (310 imp) → Shopping (CTR 2.1%)
vs Brand - Exact (CTR 8.4%). Budget cap może limitować Shopping.
Akcja: Sprawdź daily budget cap w Shopping. Rozważ podział po typach produktów.
```

---

## action: `create_task`

- 🤖 `title: "🔍 Weekly Account Audit — 27.03.2026"`
- ✅ Nowy format

## Podsumowanie #21

| | |
|---|---|
| **Status** | ✅ działa end-to-end — najlepiej zaprojektowany szablon AI |
| **Taski utworzone** | 1 per tydzień (zbiorczy) ✅ |
| **Znane problemy** | 1. Brak `limit` w kampanie getter (node 2). 2. `{2.X}` / `{3.X}` tablicowa interpolacja. 3. `agent_id: null`. |
| **Ocena ogólna** | Wzorcowy szablon — najlepszy prompt w zestawie: "exactly 3 findings", "specific numbers", "no padding". Dual-getter daje AI kontekst którego nie ma przy jednym datasource. |
| **Rekomendacja** | Dodać `limit: 20 sortBy: cost DESC` do kampanie getter. Wzorzec do naśladowania przy tworzeniu MS Ads wariantów. |

# Test #22 — `competitor_gap_analysis`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`, `days_of_week: ["monday"]` — nowy format ✅

---

## Węzeł 2 — Getter: `get_phrase_monitor_competitors`

- 👤 `project_id: null`
- ⚠️ Brak `limit` w getterze — `get_phrase_monitor_competitors` zwraca dane SEO bez parametru limitującego. Przy dużych monitoringach (100+ fraz, 5+ konkurentów) = setki rekordów.

**Dane (zmyślone, format PhrasMonitorCompetitorDTO):**

```
[
  { "query": "buty sportowe damskie", "domain": "kompetitor.pl", "rank_absolute": 2, "is_my_result": false, "title
  { "query": "buty sportowe damskie", "domain": "example.com", "rank_absolute": 5, "is_my_result": true, "title":
  { "query": "sneakers białe", "domain": "rywal.com", "rank_absolute": 1, "is_my_result": false, "title": "Białe s
  { "query": "adidas running damskie", "domain": "kompetitor.pl", "rank_absolute": 3, "is_my_result": false, "titl
]
```

---

## Węzeł 3 — Getter: `get_google_ads_performance`

- 👤 `connection_id: null`, `project_id: null`
- 📋 `resource: "keyword_view"`, 🤖 ostatnie 30 dni
- ⚠️ Brak `limit` — potencjalnie setki słów kluczowych

---

## Węzeł 4 — ai_agent

**Prompt jest najlepszy dla cross-source analizy:**

- Wyciąga frazy gdzie `is_my_result: false` (konkurenci, nie my)
- Cross-reference z naszymi keywords Google Ads
- Grupowanie luk po tematyce
- Ranking priorytetów na podstawie: liczba konkurentów, pozycje, intencja komercyjna

**Symulowana odpowiedź AI:**

```
🎯 Competitor Gap Analysis — 27.03.2026

GAP #1 — HIGH PRIORITY: "białe sneakersy" cluster
  • Phrase Monitor: rywal.com P1, kompetitor.pl P4 — 2 silni konkurenci
  • Google Ads: nie mamy słów kluczowych na "białe sneakersy", "sneakersy białe damskie"
  • Intent: transakcyjna (product category + color)
  • Akcja: dodaj [białe sneakersy damskie] exact, "sneakersy białe" phrase do ad group "Colors"

GAP #2 — MEDIUM: "adidas running damskie"
  • Phrase Monitor: kompetitor.pl P3 — 1 konkurent
  • Google Ads: mamy "adidas" broad, ale nie "adidas running damskie" explicit
  • Akcja: dodaj [adidas running damskie] exact match do kampanii Brand

POKRYTE FRAZY:
  • "buty sportowe damskie" — mamy pozycję 5 organicznie i bidujemy w Google Ads ✅
```

---

## Pytania i wnioski z testu

**Czy brak limitu na getterach jest problemem?** Tak — dla dużego konta z 200 frazami w Phrase Monitor i 500 słowami kluczowymi Google Ads, AI dostaje ogromny prompt. Może to obniżyć jakość analizy (hallucination rate rośnie przy długich kontekstach). Zalecany limit: 100 dla Phrase Monitor, 100 dla keywords sortBy cost.

**Czy cross-source analiza faktycznie działa?** Tak — to jeden z nielicznych szablonów gdzie AI ma realny powód do używania dwóch datasource jednocześnie. Pojedynczy getter nie pokaże "luk" — potrzebujesz obu zestawów. Wzorzec zasługuje na uznanie. ✅

---

## Podsumowanie #22

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 1 per tydzień (zbiorczy) ✅ |
| **Znane problemy** | 1. Brak `limit` na obu getterach — ryzyko przeciążenia promptu. 2. `{2.X}` / `{3.X}` tablicowa interpolacja. 3. `agent_id: null`. |
| **Ocena ogólna** | Jeden z bardziej wartościowych szablonów — unikalna logika cross-source (Phrase Monitor + Google Ads). Wymaga aktywnego Phrase Monitor co ogranicza grupę użytkowników. |
| **Rekomendacja** | Dodać `limit: 100` do obu getterów. Dodać notatkę w description o wymaganym Phrase Monitor (już jest — ✅). |

---

# Test #23 — `low_ctr_detective`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`, `days_of_week: ["monday"]` — nowy format ✅

---

## Węzeł 2 — Getter: `get_google_ads_performance`

- 👤 `connection_id: null`, `project_id: null`
- 📋 `resource: "campaign"`, 🤖 ostatnie 30 dni
- ⚠️ Brak `limit`

---

## Węzeł 3 — flow_control: `filter`

- 👤 `globals.min_impressions` = 500
- 👤 `globals.ctr_threshold` = 2 (2%)

**Warunki:** `impressions > 500 AND ctr < 2`

⚠️ **Potencjalny problem z typem danych:** `ctr` w Google Ads API może być zwracane jako wartość dziesiętna (0.02 = 2%) albo procentowa (2.0 = 2%). Jeśli API zwraca 0.02, warunek `ctr < 2` jest zawsze prawdziwy (0.02 < 2 = true), więc wszystkie kampanie przejdą. Wymaga weryfikacji formatu `ctr` w DTO.

**Dane testowe:**

- Brand - Exact: impressions=5380, ctr=0.058 (lub 5.8?) — do weryfikacji
- Non-Brand: impressions=27380, ctr=0.032
- Shopping: impressions=27440, ctr=0.078
- Remarketing: impressions=7420, ctr=0.12
- Display - Awareness: impressions=16120, ctr=0.009

Zakładając ctr jako dziesiętne (0.0X format), próg 2 zawsze przepuści wszystko. Zakładając ctr jako % (X.X format), przepuście kampanie z CTR < 2%.

---

## Węzeł 4 — ai_agent

**Prompt jest bardzo dobry:**

- Benchmarki CTR dla typów kampanii: search ~3–5%, display ~0.1–0.3%
- Hipotezy przyczyn: copy, targeting, match types, audience, ad scheduling
- Ranking po "wasted impressions budget"
- "Give one specific thing to check or test first"

**Symulacja AI (zakładając że Display przeszło filtr):**

```
👁️ Low CTR Audit

• Display - Awareness (URGENT): CTR 0.9%, 16,120 imp, 420 PLN

  Benchmark dla Display: 0.1–0.3%. CTR 0.9% to powyżej benchmarku display,
  ale bardzo niski dla search intencji.
  Hipoteza: błędna klasyfikacja — jeśli to kampania Display serving na search intencji,
  CTR będzie nienaturalnie niski.
  Sprawdź: jaki inventory type — Display Network czy Search?
  Jeśli Display — CTR 0.9% jest OK. Jeśli Search — masz problem z ad copy.
```

---

## Pytania i wnioski z testu

**Czy próg CTR 2% jest odpowiedni dla wszystkich typów kampanii?** Nie — CTR benchmarki znacząco różnią się między typami: Search 3-5%, Shopping 0.5-2%, Display 0.1-0.3%, Remarketing 0.3-0.5%. Jeden próg 2% jest zbyt wysoki dla Shopping/Display i za niski dla Brand Search. Idealnie: osobne progi per typ kampanii. Prompt AI to uwzględnia (podaje benchmarki) — AI jest inteligentniejszy niż sam filtr.

---

## Podsumowanie #23

| | |
|---|---|
| **Status** | ⚠️ działa, ale z potencjalnym problemem formatu CTR |
| **Taski utworzone** | 1 per tydzień (zbiorczy) ✅ |
| **Znane problemy** | 1. Format `ctr` w DTO (dziesiętne vs %) — filtr może być zawsze prawdziwy lub zawsze fałszywy. 2. Jeden próg CTR dla wszystkich typów kampanii. 3. Brak `limit` w getter. 4. `agent_id: null`. |
| **Ocena ogólna** | Dobry pomysł i dobry prompt, ale wymaga weryfikacji formatu CTR. Prompt AI mądrze uwzględnia różne benchmarki dla typów kampanii. |
| **Rekomendacja** | Zweryfikować format `ctr` w `get_google_ads_performance` response DTO. Dodać `limit: 30`. |

---

# Test #24 — `ad_copy_quality_scorer`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: monthly`, `days_of_month: "first_day"`, `time: "08:00"` — nowy format ✅

---

## Węzeł 2 — Getter: `get_google_ads_ad_copy`

- 👤 `connection_id: null`, `project_id: null`
- 📋 `limit: 500` — wysoki limit (porównaj z #15 który nie ma limitu)

⚠️ Limit 500 to bardzo dużo danych dla jednego promptu AI. Przy 500 reklamach prompt może być zbyt długi. Realny limit dla sensownej analizy to 50–100 reklam.

---

## Węzeł 3 — flow_control: `filter`

- 📋 `adStatus = "ENABLED"` — tylko aktywne reklamy ✅

---

## Węzeł 4 — ai_agent

### Prompt jest bardzo szczegółowy:

- 5 kryteriów oceny: relevance, benefit focus, CTA, wasted space, keyword alignment
- Skala 1–5 per reklama
- Trzy sekcje: NEEDS REWRITE, WATCH LIST, GOOD (z liczebnością)
- "Today is {NOW()}" — kontekst daty (pomocny przy sezonowym ad copy)

⚠️ Prompt używa `{2.X}` (getter przed filtrem). Po filtrze powinno być `{3.X}`.

### Dane (2 ENABLED reklamy z naszego przykładu):

AI analizuje:

```
Campaign: Brand, Ad Group: Nike
H1: Nike Air Max – Sale | H2: Darmowa Dostawa | H3: Letnia Wyprzedaż -30%
D1: Sprawdź letnie nowości Nike. | D2: Zamów dziś, dostawa jutro!
URL: example.com/nike
```

### Symulowana odpowiedź AI:

```
📋 Ad Copy Quality Audit – March 2026
Analyzed: 2 ads (2 NEEDS REWRITE, 0 WATCH LIST, 0 GOOD)


NEEDS REWRITE (Score: 2/5):
Brand / Nike:
❌ H3 "Letnia Wyprzedaż -30%" – marzec 2026, nie ma lata. Outdated seasonal.
❌ D1 "Sprawdź letnie nowości" – ten sam problem.
✅ H1, H2 są dobre.
Fix: Zmień H3 → "Bestsellery Wiosna 2026" i D1 → "Odkryj nowości sezonu"


Shopping / All Products:
⚠️ H1 "Kupuj Teraz" – generic, nie opisuje produktu
⚠️ H2 "Świetne Ceny" – niski benefit (każdy ma "świetne ceny")
⚠️ Brak CTA w opisach
Fix: H1 → "Buty Sportowe – Szeroki Wybór", H2 → "Darmowy Zwrot 30 Dni", D1 dodaj "Zamów do [DATA] – dostawa następ


GOOD: 0 reklam
```

---

### Różnica vs #15 (`seasonal_ad_copy_check`)

| Cecha | #15 seasonal_ad_copy_check | #24 ad_copy_quality_scorer |
|---|---|---|
| Fokus | Sezonowość i compliance | Ogólna jakość copy |
| Wzorzec | Iterator → AI per reklama | Filtr → AI dla wszystkich |
| Taski | Task per reklama | 1 zbiorczy task |
| Skala | 1 task / aktywna reklama | 1 task / miesiąc |

**#24 ma lepszy wzorzec** — zbiorczy task zamiast task per reklama. ✅

---

### Podsumowanie #24

| | |
|---|---|
| **Status** | ✅ działa end-to-end — wzorcowy pattern zbiorczego taska |
| **Taski utworzone** | 1 per miesiąc (zbiorczy) ✅ |
| **Znane problemy** | 1. `limit: 500` zbyt duże dla promptu AI — rekomendowany limit: 50–100. 2. `{2.X}` zamiast `{3.X}`. 3. `agent_id: null`. |
| **Ocena ogólna** | Jeden z najlepszych wzorców AI: miesięczna częstotliwość, zbiorczy task, precyzyjna skala ocen, trzy sekcje outputu (NEEDS REWRITE / WATCH LIST / GOOD). |
| **Rekomendacja** | Zmniejszyć `limit: 500` → `limit: 100`. Naprawić referencję `{2.X}` → `{3.X}`. |

---

# Test #25 — `ad_copy_message_audit`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: monthly`, `time: "09:00"` — nowy format ✅ (różna godzina niż #24 — 09:00 vs 08:00, żeby nie startować równocześnie)

---

## Węzeł 2–3 — Getter + Filter

- Identyczna konfiguracja jak #24: `get_google_ads_ad_copy`, `limit: 500`, filtr `adStatus = "ENABLED"`

---

## Węzeł 4 — ai_agent

**Prompt koncentruje się na czym innym niż #24:**

- **Message alignment** — czy copy pasuje do intencji kampanii (np. kampania "Brand" z generycznym copy)
- **Missing USP** — brak unikalnej propozycji wartości
- **Generic copy** — copy wymienne z dowolnym konkurentem
- **Landing page disconnect** — URL sugeruje inny temat niż nagłówki
- Grupowanie wyników **po kampanii** (a nie per reklama) — dobry wzorzec

⚠️ Ten sam problem: `{2.X}` zamiast `{3.X}`.

**Symulowana odpowiedź AI:**

```
💬 Ad Copy Message Audit — March 2026

Campaign: Brand
Issue: Ad Group "Nike" — Missing USP
  • H2 "Darmowa Dostawa" — benefit, ale nie brandowy differentiator dla Nike
  • H3 "Letnia Wyprzedaż -30%" — sezonowy, nie brandowy

Suggestion: Dodaj H "Oryginalne Nike — Oficjalny Sklep" żeby wzmocnić brand authority

Campaign: Shopping
Issue: Ad Group "All Products" — Generic copy
  • H1 "Kupuj Teraz", H2 "Świetne Ceny" — interchangeable z każdym sklepem online
  • Brak jakiegokolwiek produktu, marki, ani differentiator

Suggestion: H1 → "Buty Nike Adidas Puma", H2 → "[Rok] Kolekcja — Darmowy Zwrot"
```

---

## Różnica vs #24 (`ad_copy_quality_scorer`)

| Cecha | #24 Quality Scorer | #25 Message Audit |
|---|---|---|
| Pytanie | "Jak dobra jest ta reklama?" | "Czy reklama pasuje do kampanii?" |
| Output | Skala 1–5, NEEDS REWRITE/WATCH/GOOD | Problemy grupowane po kampanii |
| Komplementarność | Ocena jakości technicznej | Ocena spójności strategicznej |

Oba są wartościowe i różne — nie ma duplicatu. Użytkownik może mieć oba aktywne. ✅

---

## Podsumowanie #25

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 1 per miesiąc (zbiorczy) ✅ |
| **Znane problemy** | Te same co #24: `limit: 500` zbyt duże, `{2.X}` zamiast `{3.X}`, `agent_id: null`. |
| **Ocena ogólna** | Komplementarny do #24 — różny kąt analizy (strategiczna spójność vs jakość techniczna). Razem tworzą kompletny miesięczny audyt ad copy. Grupowanie po kampanii jest lepszym wzorcem niż per reklama. |
| **Rekomendacja** | Zmniejszyć `limit: 500` → `limit: 100`. Naprawić referencję. Oba szablony (#24 i #25) warto promować jako parę. |

---

# Test #26 — `ms_cpa_campaign_alert`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`, `days_of_week: ["monday"]` — nowy format ✅

---

## Węzeł 2 — Getter: `get_microsoft_advertising_campaigns`

- 👤 `connection_id: null`, `project_id: null`
- 🤖 Ostatnie 30 dni

**Dane (zmyślone, format MicrosoftAdsDTO):**

```
[
  { "campaignId": "3001", "campaignName": "Brand - Bing", "spend": 890.50, "clicks": 234, "impressions": 5710, "co
  { "campaignId": "3002", "campaignName": "Non-Brand - Bing", "spend": 2340.20, "clicks": 680, "impressions": 2345
  { "campaignId": "3003", "campaignName": "Shopping - Bing", "spend": 45.30, "clicks": 18, "impressions": 890, "co
]
```

---

## Węzeł 3 — flow_control: `filter`

- 👤 `globals.min_spend` = 100
- 📋 `conversions > 0`
- 👤 `globals.cpa_target` = 50
- Warunki: `spend > 100 AND conversions > 0 AND costPerConversion > 50`

Obliczenia:

- Brand - Bing: spend=890 > 100 ✅, conv=28 > 0 ✅, CPA=31.80 > 50 ❌ → odpada (CPA OK)
- Non-Brand - Bing: spend=2340 > 100 ✅, conv=19 > 0 ✅, CPA=123.17 > 50 ✅ → **PRZECHODZI**
- Shopping - Bing: spend=45.30 ≤ 100 ❌ → odpada (za mały spend)

**Wynik:** 1 kampania przechodzi (Non-Brand - Bing).

---

## action: `create_task`

**Task:**

> ***Tytuł:*** *`🚨 MS CPA Alert: Non-Brand - Bing — CPA: 123.17 (cel: 50)`*

>

> ***Opis:***

> *Kampania: Non-Brand - Bing (ID: 3002)*

> *Wyniki 25.02.2026 – 27.03.2026: Koszt: 2340.20 | Konwersje: 19 | Wartość: 3744 | CPA: 123.17 | ROAS: 1.6*

> *Cel CPA: 50 | Przekroczenie: +73.17*

> *Kliknięcia: 680 | Wyświetlenia: 23450 | CTR: 2.9%*

> *Sugerowane działania: sprawdź stawki, wykluczenia słów kluczowych, jakość landing page.*

✅ Task jest kompletny i actionable.

## Porównanie z #1 (`cpa_campaign_alert` Google Ads)

| Cecha | #1 Google Ads CPA | #26 MS Ads CPA |
|---|---|---|
| Format | Stary (cron) | Nowy (schedule) ✅ |
| Progi | Hardcoded | Konfigurowalny ✅ |
| `conversions > 0` | ❌ nie ma | ✅ jest (zapobiega dzieleniu przez 0) |
| Opis taska | Podstawowy | Bogaty ✅ |
| `costPerConversion` | Obliczane w filtrze | Bezpośrednie pole DTO ✅ |

MS Ads wersja jest znacząco lepsza niż Google Ads odpowiednik.

---

## Podsumowanie #26

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Taski utworzone** | 1 (Non-Brand - Bing przekracza CPA) |
| **Znane problemy** | 1. Brak deduplikacji — alert ponownie przy następnym uruchomieniu jeśli problem nie naprawiony. 2. `costPerConversion: null` dla kampanii z 0 konwersjami może powodować problemy w task description. |
| **Ocena ogólna** | Jeden z najlepiej zaprojektowanych alertów — konfigurowalny, nowy format, bogaty opis taska, zabezpieczony przed dzieleniem przez zero. Wzorzec do naśladowania przy naprawie #1. |
| **Rekomendacja** | Gotowy do użycia. Opcjonalnie: dodać zabezpieczenie na `null` costPerConversion w task title. |

---

# Test #27 — `ms_roas_drop_alert`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `frequency: weekly`, `days_of_week: ["monday"]` — nowy format ✅

---

## Węzły 2–3 — Dual Getter: dwa okresy 7-dniowe

- Węzeł 2: ostatnie 7 dni (bieżący tydzień)
- Węzeł 3: dni 8–14 (poprzedni tydzień)
- 👤 `connection_id`, `project_id` w obu — null, do uzupełnienia

✅ Wzorzec dual-getter — odpowiednik najlepszego szablonu Google Ads (#2 roas_drop_alert).

---

## Węzeł 4 — flow_control: `filter`

- 👤 `globals.min_spend` = 100
- 👤 `globals.roas_target` = 3.0

**Warunki:** `spend > 100 AND roas < 3.0` (dane z node 2 — bieżący tydzień)

Obliczenia:

- Brand - Bing: spend=890 > 100 ✅, roas=3.8 > 3.0 ❌ → odpada (ROAS OK)
- Non-Brand - Bing: spend=2340 > 100 ✅, roas=1.6 < 3.0 ✅ → PRZECHODZI

---

## action: `create_task`

**Task:**

> ***Tytuł:*** *`📉 MS ROAS Alert: Non-Brand - Bing — ROAS: 1.6 (cel: 3.0)`*

>

> ***Opis:***

> *Bieżący tydzień: Koszt 2340.20 | Wartość 3744 | Konwersje: 19 | ROAS: 1.6 | CTR: 2.9%*

> *Poprzedni tydzień: Koszt 2150 | Wartość 3870 | ROAS: 1.8*

> *Cel ROAS: 3.0 | Zmiana ROAS: -11.1%*

> *Sprawdź tracking konwersji, stawki i sezonowość.*

⚠️ **Nazewnictwo "ROAS Drop Alert" jest mylące podobnie jak #2.** ROAS wynosi 1.6, zawsze był niski — nie jest to "drop". Workflow wykrywa ROAS poniżej progu, nie spadek. Sama logika (dual-getter + porównanie) jest wzorcowa, ale nazwa sugeruje coś innego.

---

## Podsumowanie #27

| | |
|---|---|
| **Status** | ✅ działa end-to-end — analogiczny do #2 (Google Ads) |
| **Taski utworzone** | 1 (Non-Brand poniżej ROAS) |
| **Znane problemy** | 1. Nazwa "Drop Alert" myląca — wykrywa próg, nie trendu. 2. Brak deduplikacji. |
| **Ocena ogólna** | Porównywalny z #2 (roas_drop_alert) — nowy format, konfigurowalny, dual-getter dla porównania historycznego. Wzorcowy szablon alertowy. |
| **Rekomendacja** | Gotowy do użycia. Rozważyć zmianę nazwy na "ROAS Below Target" dla precyzji. |

# Test #28 — `ms_zero_impression_campaigns`

**Data testu:** 2026-03-27

Analogiczny do #3 (`zero_impression_campaigns` Google Ads) — ta sama logika, inne źródło danych.

### Węzeł 1 — Trigger: weekly, poniedziałek, nowy format ✅

### Węzeł 2 — Getter: `get_microsoft_advertising_campaigns`, ostatnie 7 dni

### Węzeł 3 — Filter: `impressions < 1`

### Węzeł 4 — Iterator po wyfiltrowanych kampaniach

### Węzeł 5 — create_task (nowy format, project_id: null ✅)

**Dane testowe:**

```
[
  { "campaignName": "Brand - Bing", "impressions": 5710, "campaignStatus": "ACTIVE" },
  { "campaignName": "New Spring Campaign", "impressions": 0, "campaignStatus": "ACTIVE", "clicks": 0, "spend": 0 }
]
```

Filtr: Brand-Bing (5710 ≥ 1) → odpada. New Spring Campaign (0 < 1) → PRZECHODZI.

**Task:**

> ***Tytuł:*** *`👻 MS Zero wyświetleń: New Spring Campaign (ostatnie 7 dni)`*

> ***Opis:*** *Status: ACTIVE | Wyświetlenia: 0 | Lista przyczyn do sprawdzenia...*

✅ Dobry opis z listą diagnoz. Przewaga nad Google Ads odpowiednikiem (#3): zawiera `{2.campaignStatus}` w opisie.

## Podsumowanie #28

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Znane problemy** | Brak deduplikacji — te same kampanie w każdym tygodniu |
| **Ocena ogólna** | Lepsza wersja #3 — nowy format, bogatszy opis, konfigurowalny. Wzorzec dla naprawy Google Ads odpowiednika. |

# Test #29 — `ms_budget_protector`

**Data testu:** 2026-03-27

Odpowiednik #4 (`budget_protector` Google Ads) — **znacząco lepszy**.

## Kluczowe różnice vs #4

| Cecha | #4 Google Ads | #29 MS Ads |
|---|---|---|
| Trigger interval | co 1h (stary cron) | co 4h (nowy format) ✅ |
| Próg | hardcoded `cost > 100` | `globals.daily_budget_threshold` ✅ |
| Format | stary | nowy ✅ |
| Opis taska | podstawowy | bogaty z CPA ✅ |
| `globals` | `[]` (tablica!) | obiekt z `daily_budget_threshold: 500` ✅ |

## Węzeł 1 — Trigger: `interval: 4` (co 4h) — nowy format ✅

## Węzeł 2 — Getter: `get_microsoft_advertising_campaigns`, dzisiaj (start_date = end_date = NOW)

## Węzeł 3 — Filter: `spend > {globals.daily_budget_threshold}` (domyślnie 500)

## Węzeł 4 — Iterator

## Węzeł 5 — create_task z CPA, CTR, konwersjami

**Dane testowe (dzisiaj):**

```
{ "campaignName": "Non-Brand - Bing", "spend": 680, "clicks": 195, "conversions": 5, "ctr": 0.024, "costPerConvers
```

Filtr: spend=680 > 500 → PRZECHODZI.

**Task:**

> ***Tytuł:*** *`🔴 MS Budget Alert: Non-Brand - Bing — 680 dzisiaj (próg: 500)`*

> ***Opis:*** *Rzeczywiste: 680 | Próg: 500 | Kliknięcia: 195 | Konwersje: 5 | CPA: 136...*

⚠️ Problem nadal istnieje: brak deduplikacji. Co 4h = 6 uruchomień/dobę. Przy aktywnym przekroczeniu: 6 tasków/dobę per kampania. Lepiej niż 24/dobę (#4), ale wciąż potencjalny task flood.

## Podsumowanie #29

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Znane problemy** | Brak deduplikacji — co 4h alert dla tej samej kampanii dopóki problem nie naprawiony |
| **Ocena ogólna** | Znacznie lepszy niż Google Ads odpowiednik — nowy format, konfigurowalny próg, co 4h zamiast co 1h. Nadal brak mechanizmu deduplikacji. |
| **Rekomendacja** | Wzorzec do naśladowania przy naprawie #4. Dodać notatkę w opisie szablonu o potencjalnym duplikowaniu alertów. |

# Test #30 — `ms_budget_deviation`

**Data testu:** 2026-03-27

Odpowiednik #5 (`budget_deviation`) — ale w przeciwieństwie do #5, to jest **prawdziwy template**.

## Węzeł 1 — Trigger

- 📋 `frequency: daily`, `time: "20:00"` — nowy format ✅
- Codziennie o 20:00 — sensowna godzina (koniec dnia, dane są kompletne)

## Węzeł 2 — Getter: `get_microsoft_advertising_campaigns`

- 🤖 `start_date: {NOW().getFirstDayCurrentMonth()}` — od początku bieżącego miesiąca
- 🤖 `end_date: {NOW()}` — do dziś

⚠️ Getter pobiera WSZYSTKIE kampanie od początku miesiąca — filtr `spend > monthlyDeviation` dotyczy per kampania, nie całego konta. Jeśli użytkownik ma budżet 5000 PLN dla całego konta ale 5 kampanii, każda kampania jest sprawdzana osobno względem pełnego budżetu.

Logicznie: to powinno być sprawdzenie na poziomie konta (suma spend wszystkich kampanii), nie per kampania. Każda kampania może być daleka od 5000 PLN osobno, ale razem mogą przekraczać próg.

## Węzeł 3 — Filter: `spend > monthlyDeviation(5000, 10)`

Dla 27. dnia marca (31 dni): `5000 × (27/31) × 1.10 = 4763.7`

Kampania Non-Brand - Bing z miesięcznym spend = 8200 PLN:

- `8200 > 4763.7` → PRZECHODZI (alert!)

Ale suma wszystkich kampanii = problem do sprawdzenia na poziomie konta, nie kampanii.

## action: `create_task`

**Task:**

> ***Tytuł:*** *`MS Budżet na dzień 27 miesiąca przekroczył 10% odchylenia`*

> ***Opis:*** *Limit: 5000 | Wydatki do 27.03: 8200 | Oczekiwane: 4330 | Odchylenie: +89.4% | Pozostały budżet: -3200 | Rekomendowany dzienny: -533 (ujemny = wyczerpany)*

✅ Opis taska z ujemnym "pozostałym budżetem" prawidłowo sygnalizuje przekroczenie.

---

## Podsumowanie #30

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Znane problemy** | 1. Logika per-kampania zamiast per-konto — alert może nie wykryć przekroczenia łącznego budżetu jeśli żadna kampania indywidualnie nie przekracza progu. 2. Brak deduplikacji (alert codziennie jeśli przekroczenie trwa przez wiele dni). |
| **Ocena ogólna** | Znacznie lepsza wersja #5 (prawdziwy template, brak hardcoded ID). Logika `monthlyDeviation` jest wzorcowa. Problem granularności (per kampania vs konto). |
| **Rekomendacja** | Rozważyć dodanie wariantu z `get_microsoft_advertising_customers` (poziom konta) zamiast per kampania. |

---

# Test #31 — `ms_non_converting_keywords`

**Data testu:** 2026-03-27

Odpowiednik #9 (`non_converting_keywords` Google Ads) — **lepsza wersja**.

## Kluczowe różnice vs #9

| Cecha | #9 Google Ads | #31 MS Ads |
|---|---|---|
| Format | Stary (cron) | Nowy ✅ |
| Próg | Hardcoded `cost > 20` | `globals.min_spend = 50` ✅ |
| Getter | `get_google_ads_performance keyword_view` | `get_microsoft_advertising_keywords` |
| Opis taska | Podstawowy | Bogaty (keyword, keywordId, avgCPC, suggestions) ✅ |
| `{2.keyword}` | `{4.title}` (po iteratorze) | `{2.keyword}` ✅ |

**Dane testowe:**

```
[
  { "keyword": "buty ortopedyczne", "keywordId": "9001", "campaignName": "Non-Brand", "adGroupName": "Footwear", "
  { "keyword": "adidas running damskie", "spend": 120.30, "conversions": 3 }
]
```

Filtr: "buty ortopedyczne" (spend=87.5 > 50 ✅, conv=0 < 1 ✅) → PRZECHODZI. Adidas (conv=3) → odpada.

**Task:**

> ***Tytuł:*** *`🚫 MS Non-converting: 'buty ortopedyczne' — 87.5 spend, 0 konwersji (60 dni)`*

> ***Opis:*** *Słowo: buty ortopedyczne | Kampania: Non-Brand | Koszt: 87.5 | Avg CPC: 2.57 | Lista sugestii...*

✅ Bogaty, actionable task z konkretnym słowem kluczowym.

⚠️ Ten sam problem skalowalności co #9 — task per słowo kluczowe. 50 niekonwertujących słów = 50 tasków/tydzień.

## Podsumowanie #31

| | |
|---|---|
| **Status** | ✅ działa end-to-end — lepsza wersja #9 |
| **Znane problemy** | Task per słowo = problem skalowalności przy dużych kontach |
| **Ocena ogólna** | Wzorcowy szablon alertowy MS Ads — nowy format, konfigurowalny próg, bogaty opis taska. |

# Test #32 — `ms_trending_search_terms`

**Data testu:** 2026-03-27

Odpowiednik #10 (`trending_search_terms` Google Ads) — z **pewnymi ulepszeniami**.

## Kluczowe różnice vs #10

| Cecha | #10 Google Ads | #32 MS Ads |
|---|---|---|
| Format | Stary (cron) | Nowy ✅ |
| Próg | Hardcoded `impressions > 100` | `globals.min_impressions = 100` ✅ |
| Getter | + `limit: 20 sortBy: impressions` | Brak limitu i sortowania |
| Pola taska | `title`, `campaign_name` | `searchQuery`, `keyword`, `campaignName`, `roas` ✅ |

⚠️ Brak `limit` i `sortField` w MS Ads getter — może zwrócić dużo danych bez sortowania. W Google Ads #10 getter zwraca TOP 20 po wyświetleniach, co ma sens. Tu nie ma limitu.

⚠️ Ten sam problem nazewnictwa — "Trending" to popularne, nie rosnące. Opis szablonu mówi "dużą liczbą wyświetleń i dobrym CTR" — to uczciwe, ale nazwa wciąż sugeruje trend wzrostowy.

**Dane testowe:**

```
[
  { "searchQuery": "nike air max bing", "keyword": "nike air max", "campaignName": "Brand", "adGroupName": "Nike",
]
```

**Task:**

> ***Tytuł:*** *`📈 MS Trending: 'nike air max bing' — 520 wyświetleń, CTR: 8.1%`*

> ***Opis:*** *Fraza: nike air max bing | Dopasowane kw: nike air max | Kampania: Brand | CTR: 8.1% | Konwersje: 5 | ROAS: 4.2 | Rozważ dodanie jako exact match...*

✅ Opis jest bogatszy niż #10 — zawiera `searchQuery` (fraza użytkownika) oddzielne od `keyword` (nasze słowo), plus ROAS.

## Podsumowanie #32

| | |
|---|---|
| **Status** | ✅ działa end-to-end |
| **Znane problemy** | 1. Brak `limit` i `sortField` w getter. 2. Nazwa "Trending" myląca. 3. Task per fraza = skalowalność. |
| **Ocena ogólna** | Lepsza wersja #10 — konfigurowalny próg, bogatszy opis taska. Nowy format. Drobne braki do uzupełnienia. |

# Test #33 — `ms_ai_weekly_report`

**Data testu:** 2026-03-27

Odpowiednik #12 (`ai_weekly_report` Google Ads). Identyczna struktura — różny getter.

## Różnice vs #12

| Cecha | #12 Google Ads | #33 MS Ads |
|---|---|---|
| Format | Stary (cron) | Nowy ✅ |
| Getter | `get_google_ads_performance resource:campaign` | `get_microsoft_advertising_campaigns` |
| CPA w prompcie | `{2.cost}/{2.conversions}` (tekst) | `{2.costPerConversion}` (bezpośrednie pole) ✅ |
| Dane | cost, clicks, conv, ctr | spend, clicks, conv, ctr, roas, costPerConversion ✅ |
| create_task | Stary format | Nowy format ✅ |

MS Ads getter bezpośrednio zwraca `costPerConversion` i `roas` — Google Ads wymaga obliczenia. To upraszcza prompt. ✅

## Podsumowanie #33

| | |
|---|---|
| **Status** | ✅ działa end-to-end (z tym samym zastrzeżeniem co #12: {2.X} może być tylko pierwszym elementem) |
| **Taski utworzone** | 1 per tydzień ✅ |
| **Znane problemy** | 1. `{2.X}` w prompcie — ryzyko tablicowej interpolacji. 2. `agent_id: null`. |
| **Ocena ogólna** | Lepsza wersja #12 — nowy format, bezpośrednie pola CPA/ROAS. Wzorzec: zbiorczy task + nowy format. |

# Test #34 — `ms_conversion_comparison`

**Data testu:** 2026-03-27

Odpowiednik #11 (`conversion_comparison`) — **prawdziwy template** (w przeciwieństwie do #11 który był prawdziwym workflow użytkownika).

## Kluczowe różnice vs #11

| Cecha | #11 Google Ads | #34 MS Ads |
|---|---|---|
| To template? | ❌ Prawdziwy workflow użytkownika | ✅ Prawdziwy template |
| Hardcoded IDs | ❌ connection_id=652, project_id=909 | ✅ null (do uzupełnienia) |
| Trigger | Hourly (!) | Weekly ✅ |
| `min_spend` | Brak | `globals.min_spend = 100` ✅ |
| Opis | Niezgodny z logiką | Zgodny ✅ |

## Węzły 2–3 — Dual Getter: dwa okresy 30-dniowe

Identyczna logika jak #11 — ostatnie 30 dni (node 2) vs poprzednie 30 dni (node 3).

## Węzeł 4 — Filter

Warunki: `spend > 100 AND percentageChange(conv_obecne, conv_poprzednie) < -5`

**Dane testowe:**

- Non-Brand - Bing: spend_30d=2340, conv_30d=19, conv_poprz=28
- `percentageChange(19, 28) = (19-28)/28×100 = -32.1%`
- `-32.1% < -5%` → ✅ PRZECHODZI

**Task:**

> ***Tytuł:*** *`MS Non-Brand - Bing: Spadek konwersji -32.1% (koszt: 2340)`*

> ***Opis:*** *Ostatnie 30 dni: 19 konwersji, 2340 spend, ROAS 1.6 | Poprzednie 30 dni: 28 konwersji, 2150 spend, ROAS 1.8 | Zmiana: -32.1% | Próg alertu: -5%*

## Podsumowanie #34

| | |
|---|---|
| **Status** | ✅ działa end-to-end — wzorcowy template alertowy |
| **Taski utworzone** | 1 per kampania poniżej progu |
| **Znane problemy** | Brak deduplikacji — alert tygodniowo dopóki problem trwa |
| **Ocena ogólna** | Znakomita implementacja w porównaniu z Google Ads odpowiednikiem (#11). Wzorzec do naśladowania przy przepisywaniu #11. |

# Test #35 — `ms_campaign_intelligence_scoring`

**Data testu:** 2026-03-27

Odpowiednik #13 (`campaign_intelligence_scoring` Google Ads). Identyczna struktura — różny getter.

## Różnice vs #13

| Cecha | #13 Google Ads | #35 MS Ads |
|---|---|---|
| Format | Stary (cron) | Nowy ✅ |
| CPA w prompcie | `{2.cost}/{2.conversions}` | `{2.costPerConversion}` ✅ |
| Revenue | Brak | `{2.revenue}` ✅ |
| Ikona w task | Brak | 🧠 ✅ |

Prompt jest niemal identyczny z #13 ale z `revenue` jako dodatkową zmienną. MS Ads dostarcza więcej pól bezpośrednio (costPerConversion, revenue, roas) vs Google Ads gdzie trzeba obliczać.

## Podsumowanie #35

| | |
|---|---|
| **Status** | ✅ działa end-to-end — lepsza wersja #13 |
| **Taski utworzone** | 1 per tydzień ✅ |
| **Znane problemy** | `{2.X}` interpolacja tablicy, `agent_id: null` |
| **Ocena ogólna** | Porównywalny z #13. Nowy format, bezpośrednie pola CPA/ROAS/revenue. |

## Test #36 — `ms_seasonal_ad_copy_check`

**Data testu:** 2026-03-27

### ⚠️ UWAGA: AI nie widzi treści reklam

Getter: `get_microsoft_advertising_campaigns` — zwraca `campaignName`, `spend`, `clicks`, `ctr`. **Zero treści reklam.**

AI prompt prosi o review sezonowości ad copy, ale AI dostaje tylko **nazwę kampanii** i metryki wydajności. AI musi zgadywać treść reklam na podstawie nazwy kampanii.

**Kontrast z #15 (`seasonal_ad_copy_check` Google Ads):**

| Cecha | #15 Google Ads | #36 MS Ads |
|---|---|---|
| Getter | `get_google_ads_ad_copy` — pełna treść reklam | `get_microsoft_advertising_campaigns` — tylko metryki |
| AI widzi | Headline1-3, Description1-2, URL | Tylko campaignName |
| Wartość | ✅ Realny review treści | ❌ AI zgaduje na podstawie nazwy |

**Przykład problemu:**

- Kampania "Non-Brand - Shoes" może mieć nagłówki "Letnia Wyprzedaż -30%"
- AI dostaje tylko "Non-Brand - Shoes, spend=2340"
- AI odpowiada: "Nazwa kampanii nie sugeruje sezonowości, sprawdź czy reklamy mają sezonowe odniesienia"
- Użytkownik myśli że AI przejrzało reklamy — nie przejrzało ani jednego nagłówka

### Węzeł 4 — Iterator + ai_agent per kampania

⚠️ Wzorzec iterator → AI → task per kampania = task per aktywna kampania. 10 kampanii = 10 tasków miesięcznie, każdy z "analizą" opartą na samej nazwie.

### Podsumowanie #36

| | |
|---|---|
| **Status** | ❌ fundamentalnie zepsuty — AI nie ma dostępu do treści reklam |
| **Taski utworzone** | 1 per aktywna kampania (N tasków/miesiąc) |
| **Znane problemy** | 1. Brak gettera treści reklam MS Ads — AI zgaduje na podstawie nazwy kampanii. 2. Myląca wartość — użytkownik sądzi że AI przejrzało reklamy. 3. Task per kampania (skalowalność). |
| **Ocena ogólna** | Dobry pomysł, zła realizacja. Musi zostać usunięty lub zawieszony do czasu gdy Microsoft Ads getter będzie zwracał treści reklam (analogicznie do `get_google_ads_ad_copy`). |
| **Rekomendacja** | Usunąć do czasu implementacji `get_microsoft_advertising_ad_copy` gettera. Wyjaśnione w audycie. |

---

# Test #37 — `microsoft_ads_waste_finder`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger: weekly, poniedziałek 09:00, nowy format ✅

## Węzeł 2 — Getter: `get_microsoft_advertising_search_terms`

- 👤 `connection_id: null`, `project_id: null`
- 🤖 Ostatnie 30 dni
- 📋 `ms_ads_raw_xml_request: null` — parametr obecny ale null

## Węzeł 3 — Filter

- 👤 `globals.min_spend` = 50
- Warunki: `spend > 50 AND conversions < 1`

## Węzeł 4 — Iterator (iteruje po {sourceId: 3})

## Węzeł 5 — create_task (nowy format, project_id: null ✅)

**Dane testowe:**

```
[
  { "searchQuery": "buty do tańca bing", "campaignName": "Non-Brand", "adGroupName": "Footwear", "keyword": "buty
  { "searchQuery": "nike air max 2026", "spend": 120.30, "conversions": 4 }
]
```

Filtr: "buty do tańca bing" (spend=89.5 > 50 ✅, conv=0 < 1 ✅) → PRZECHODZI. Nike (conv=4) → odpada.

**Task:**

> ***Tytuł:*** *`🔥 Waste: "buty do tańca bing" — 89.5 spend, 0 konwersji`*

> ***Opis:*** *Fraza: "buty do tańca bing" | Kampania: Non-Brand | Kw: buty taneczne | Wydatki: 89.50 | CTR: 3.6% | AvgCPC: 1.99 | CTR wysoki → problem na landing page | Lista 5 rekomendacji...*

✅ Bardzo bogaty, actionable task. Jeden z najlepiej opisanych tasków w całym zestawie.

**Wartość dodana:** Różnicowanie CTR wysoki (problem LP) vs CTR niski (nieistotna fraza) w instrukcjach w opisie. ✅

## Podsumowanie #37

| | |
|---|---|
| **Status** | ✅ działa end-to-end — jeden z najlepszych szablonów |
| **Taski utworzone** | 1 per przepaloną frazę (z 30 dni) |
| **Znane problemy** | Task per fraza = skalowalność przy dużych kontach. Brak limitu w getterze. |
| **Ocena ogólna** | Wzorcowy szablon "waste detection" — konfigurowalny próg, bogaty actionable opis taska z diagnostyką CTR vs LP, nowy format. Jeden z najlepszych szablonów w zestawie MS Ads. |
| **Rekomendacja** | Dodać `limit` do gettera. Opcjonalnie: zbiorczy task zamiast task per fraza. |

# Test #38 — `ms_ads_winners`

**Data testu:** 2026-03-27

## Węzeł 1 — Trigger

- 📋 `trigger_type: cron`, `cron: "0 8 1"` — **stary format**, poniedziałek 08:00

## Węzeł 2 — Getter: `get_microsoft_advertising_search_terms`

- 👤 `connection_id: ""`, `project_id: ""` — puste stringi (nie null jak w nowszych)
- ⚠️ Stary format parametrów: `startDate` / `endDate` (camelCase) vs `start_date` / `end_date` (snake_case w nowszych)
- 🤖 Ostatnie 30 dni

## Węzeł 3 — flow_control: `filter` (stary format)

- 📋 `left: "conversions"`, `operator: ">"`, `right: "2"` — hardcoded
- 📋 `left: "ctr"`, `operator: ">"`, `right: "0.05"` — hardcoded próg CTR 5%

⚠️ Próg CTR 0.05 — jeśli `ctr` jest w formacie dziesiętnym (0.036 = 3.6%), warunek `ctr > 0.05` = 5%. Jeśli `ctr` jest w formacie procentowym (3.6 = 3.6%), warunek `ctr > 0.05` przepuści WSZYSTKO (3.6 >> 0.05). Ten sam problem formatu co w #23.

⚠️ Stary format filtra (`left_source: "2"`, stara struktura warunków).

## Węzeł 5 — action: `create_task` (stary format)

- ❌ Stary format (`action_type: create_task` w `data`)
- ❌ Odwołuje się do `{4.searchTerm}` ale getter MS Ads zwraca `searchQuery` (nie `searchTerm`)

**Dane testowe:**

```
{ "searchQuery": "nike air max bing", "conversions": 5, "revenue": 650, "ctr": 0.081, "spend": 89.50 }
```

Filtr: `conversions > 2` ✅ (5 > 2), `ctr > 0.05` — zależy od formatu:

- Jeśli ctr=0.081 (dziesiętne): 0.081 > 0.05 ✅ → PRZECHODZI
- Jeśli ctr=8.1 (procentowe): 8.1 > 0.05 ✅ → PRZECHODZI

Dla tej frazy filtr przejdzie w obu interpretacjach.

**Task:**

> ***Tytuł:*** *`MS Ads winner: [null]` ← pole `{4.searchTerm}` nie istnieje, jest `searchQuery`*

> ***Opis:*** *Search term: [null], Conversions: 5, Revenue: 650, CTR: 0.081...*

❌ Błąd w tytule i opisie: `searchTerm` → powinno być `searchQuery`.

## Pytania i wnioski z testu

**Czy koncepcja "MS winners → Google Ads expansion" jest wartościowa?** Tak — cross-platform optymalizacja: frazy które działają na Bing mogą być niedoreprezentowane na Google. To dobry use case dla kont które prowadzą kampanie na obu platformach. ✅

**Czy szablon dostarcza wartość mimo błędów?** Błąd `searchTerm` / `searchQuery` jest kosmetyczny (do naprawy). Format jest stary. Koncepcja jest prawidłowa.

## Podsumowanie #38

| | |
|---|---|
| **Status** | ⚠️ działa technicznie z błędem w nazwie pola |
| **Taski utworzone** | N tasków per fraza spełniającą warunki |
| **Znane problemy** | 1. `{4.searchTerm}` → powinno być `{4.searchQuery}`. 2. Stary format triggera, filtra, create_task. 3. Hardcoded progi (conversions > 2, ctr > 0.05). 4. Format CTR niezweryfikowany (jak w #23). |
| **Ocena ogólna** | Dobra koncepcja (MS→Google expansion), wymagająca aktualizacji formatu i naprawy nazwy pola. Jeden z bardziej unikalnych szablonów pod kątem use case. |
| **Rekomendacja** | Naprawić `searchTerm` → `searchQuery`. Dodać globals dla konfigurowalnych progów. Przenieść na nowy format. |

# Podsumowanie wszystkich 38 testów

**Data podsumowania:** 2026-03-27

## Status testów

| # | Slug | Status | Kluczowy problem |
|---|---|---|---|
| 1 | `cpa_campaign_alert` | ✅ Działa | Brak filtra biddingStrategy, stary format |
| 2 | `roas_drop_alert` | ✅ Działa | Nazwa myląca (próg, nie trend) |
| 3 | `zero_impression_campaigns` | ✅ Działa | Fałszywe alarmy: nowe kampanie, ad scheduling |
| 4 | `budget_protector` | ❌ Błędny | Task flood 24×/dobę, hardcoded próg, stary format |
| 5 | `budget_deviation` | ❌ Nie template | Dane prawdziwego użytkownika (GDPR) |
| 6 | `visibility_drop_alert` | ❌ Błędny | Pola camelCase zamiast snake_case → zero tasków |
| 7 | `price_competitiveness_alert` | ✅ Działa | Ceny w mikrosach w taskach |
| 8 | `inventory_aware_alert` | ❌ Błędny | Market-level data, nieistniejące pola DTO |
| 9 | `non_converting_keywords` | ✅ Działa | Hardcoded próg, skalowalność (task/słowo) |
| 10 | `trending_search_terms` | ⚠️ Mylący | Nazwa "Trending" ≠ logic (pokazuje popularne) |
| 11 | `conversion_comparison` | ❌ Nie template | Dane prawdziwego użytkownika (GDPR) |
| 12 | `ai_weekly_report` | ⚠️ Ryzyko | {2.X} może być tylko pierwszym elementem tablicy |
| 13 | `campaign_intelligence_scoring` | ⚠️ Ryzyko | Jak #12 |
| 14 | `pmax_analysis` | ❌ Błędny | Market-level data, nieistniejące pola, błędna struktura węzłów |
| 15 | `seasonal_ad_copy_check` | ✅ Działa | Task per reklama = skalowalność |
| 16 | `daily_ppc_brief` | ✅ Działa | Prompt używa {2.X} zamiast {3.X} po filtrze |
| 17 | `weekly_opportunity_scout` | ✅ Działa | Wzorcowy szablon AI — nieznaczne uwagi |
| 18 | `search_terms_clustering` | ✅ Działa | Wzorcowy szablon AI |
| 19 | `monthly_cross_platform_summary` | ✅ Działa | Brak limit na getterach |
| 20 | `keyword_quality_audit` | ✅ Działa | Brak limit w getterze |
| 21 | `master_account_audit` | ✅ Działa | Wzorcowy prompt (3 findings, no padding) |
| 22 | `competitor_gap_analysis` | ✅ Działa | Brak limit na getterach (ryzyko dużych danych) |
| 23 | `low_ctr_detective` | ⚠️ Ryzyko | Format CTR (dziesiętne vs %) nieznany |
| 24 | `ad_copy_quality_scorer` | ✅ Działa | limit: 500 zbyt duże, {2.X} zamiast {3.X} |
| 25 | `ad_copy_message_audit` | ✅ Działa | Jak #24 |
| 26 | `ms_cpa_campaign_alert` | ✅ Działa | Wzorcowy alert MS Ads |
| 27 | `ms_roas_drop_alert` | ✅ Działa | Nazwa myląca (próg, nie trend) |
| 28 | `ms_zero_impression_campaigns` | ✅ Działa | Brak deduplikacji |
| 29 | `ms_budget_protector` | ✅ Działa | Brak deduplikacji (6×/dobę) |
| 30 | `ms_budget_deviation` | ✅ Działa | Per-kampania zamiast per-konto |
| 31 | `ms_non_converting_keywords` | ✅ Działa | Task per słowo = skalowalność |
| 32 | `ms_trending_search_terms` | ✅ Działa | Brak limit/sort, nazwa myląca |
| 33 | `ms_ai_weekly_report` | ✅ Działa | {2.X} ryzyko tablicy |
| 34 | `ms_conversion_comparison` | ✅ Działa | Wzorcowy template alertowy |
| 35 | `ms_campaign_intelligence_scoring` | ✅ Działa | {2.X} ryzyko tablicy |
| 36 | `ms_seasonal_ad_copy_check` | ❌ Bezużyteczny | AI nie widzi treści reklam (brak gettera) |

| | | | |
|---|---|---|---|
| 37 | `microsoft_ads_waste_finder` | ✅ Działa | Brak limit w getterze |
| 38 | `ms_ads_winners` | ⚠️ Błąd | `searchTerm` → `searchQuery`, stary format |

## Statystyki

- ✅ Działa end-to-end: **25 szablonów** (66%)
- ⚠️ Działa z zastrzeżeniami: **5 szablonów** (13%)
- ❌ Nie działa / nie jest templatem: **8 szablonów** (21%)

## Krytyczne do naprawy (przed udostępnieniem)

- **#4** `budget_protector` — Przebudować: nowy format, global threshold, interval 4h
- **#5** `budget_deviation` — Przepisać od zera: usunąć GDPR, hardcoded ID, naprawić trigger
- **#6** `visibility_drop_alert` — Naprawić: camelCase → snake_case w polach filtra
- **#8** `inventory_aware_alert` — Usunąć lub przepisać: zły getter + nieistniejące pola
- **#11** `conversion_comparison` — Przepisać od zera: usunąć GDPR, hardcoded ID
- **#14** `pmax_analysis` — Usunąć lub przepisać: market-level data + błędna architektura
- **#36** `ms_seasonal_ad_copy_check` — Usunąć: AI nie widzi treści reklam MS Ads

## Wzorcowe szablony (punkty odniesienia)

- **#2** `roas_drop_alert` — Najlepszy szablon alertowy Google Ads (nowy format, konfigurowalny, dual-getter)
- **#17** `weekly_opportunity_scout` — Najlepszy szablon AI weekly (zbiorczy task, dobry prompt, nowy format)
- **#21** `master_account_audit` — Najlepszy prompt AI (exactly 3 findings, no padding)
- **#26** `ms_cpa_campaign_alert` — Najlepszy szablon alertowy MS Ads
- **#37** `microsoft_ads_waste_finder` — Najlepszy opis taska MS Ads (bogaty, diagnostyczny)

---